Dynamics Of Celestial Bodies

Numerical Solutions With C++

Taylor Larrechea - March 12, 2026

About Me

Academic

- Graduated from CMU with B.S in Physics and Minor in Mathematics in 2020
- Attended graduate school at The University of Oklahoma from 2021 to 2022
- Graduated from CU Boulder with B.S in Computer Science in 2024
- Published paper to APS with Dr. David Collins in December 2025

Professional

- Formulation Scientist at Mesa Lavender Farms from June 2020 to February 2021
- Customer Engineer at Applied Materials from October 2022 to May 2025
- Junior iOS Engineer as contractor at Vivint from June 2025 - Present

Agenda

1. Background Of Project
2. Motivation For Refactoring
3. The Math
4. The Physics
5. Refactored Results
6. Physics Informed Neural Network
7. Conclusions
8. Questions

Background Of Project

Advanced Dynamics - Fall Semester Of 2018

- Completed basic implementation of numerical solver with FEDS numerical method
- Presented results in Fall Semester of 2019 at CMU

GUI Implementation With RK4 - Fall Semester Of 2023

- Created graphical user interface with RK4 solver

C++ Implementation With RK4 - November 2025 to March 2026

- Refactored numerical solvers to be in C++ instead of Python
  - With persistent data
- Created N-Body for (hypothetically) infinite number of bodies in system
- Created framework for Machine Learning model training from data

Motivation For Refactoring

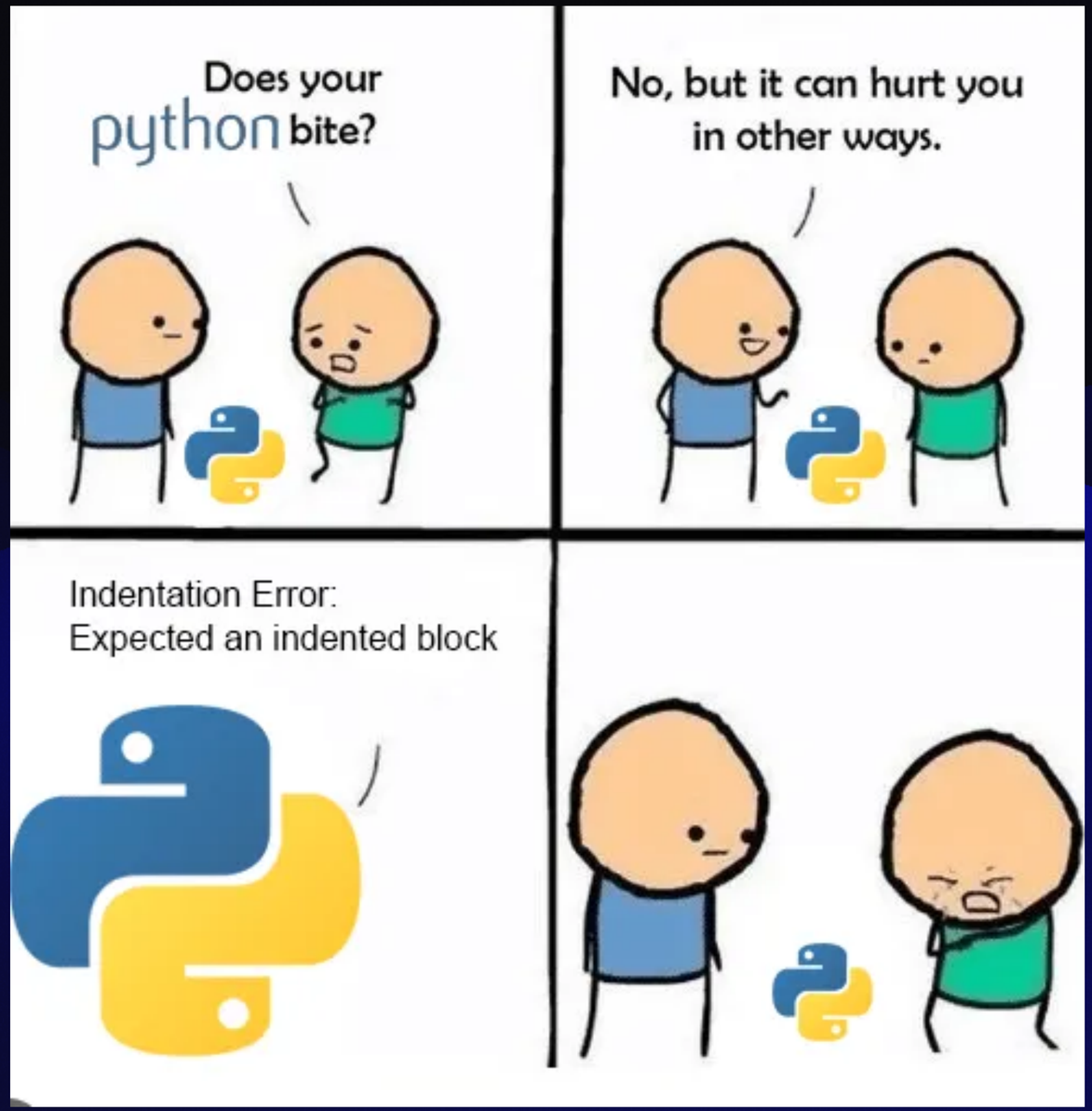
Initial Refactoring

- The Forward Euler Difference Scheme (FEDS) requires a very small time step
  - This can lead to very long computation times due to small time steps
- The Runge Kutta 4 (RK4) numerical method can take in a smaller time step
  - This leads to less computation time and faster solution times
- Python is very, very slow compared to a compiled language like C++
  - Interpreted language : Writing a book report page by page , Compiled language : Writing a book report after reading the entire book
- Persistent data production to be able to review previous computations
  - Previous implementations did not have the ability to review previous computations
- Fast and large data production for ML model training

Let’s Play A Game

Python vs. C++

- Two applications written that count to 1 billion
- One in Python
- One in C++



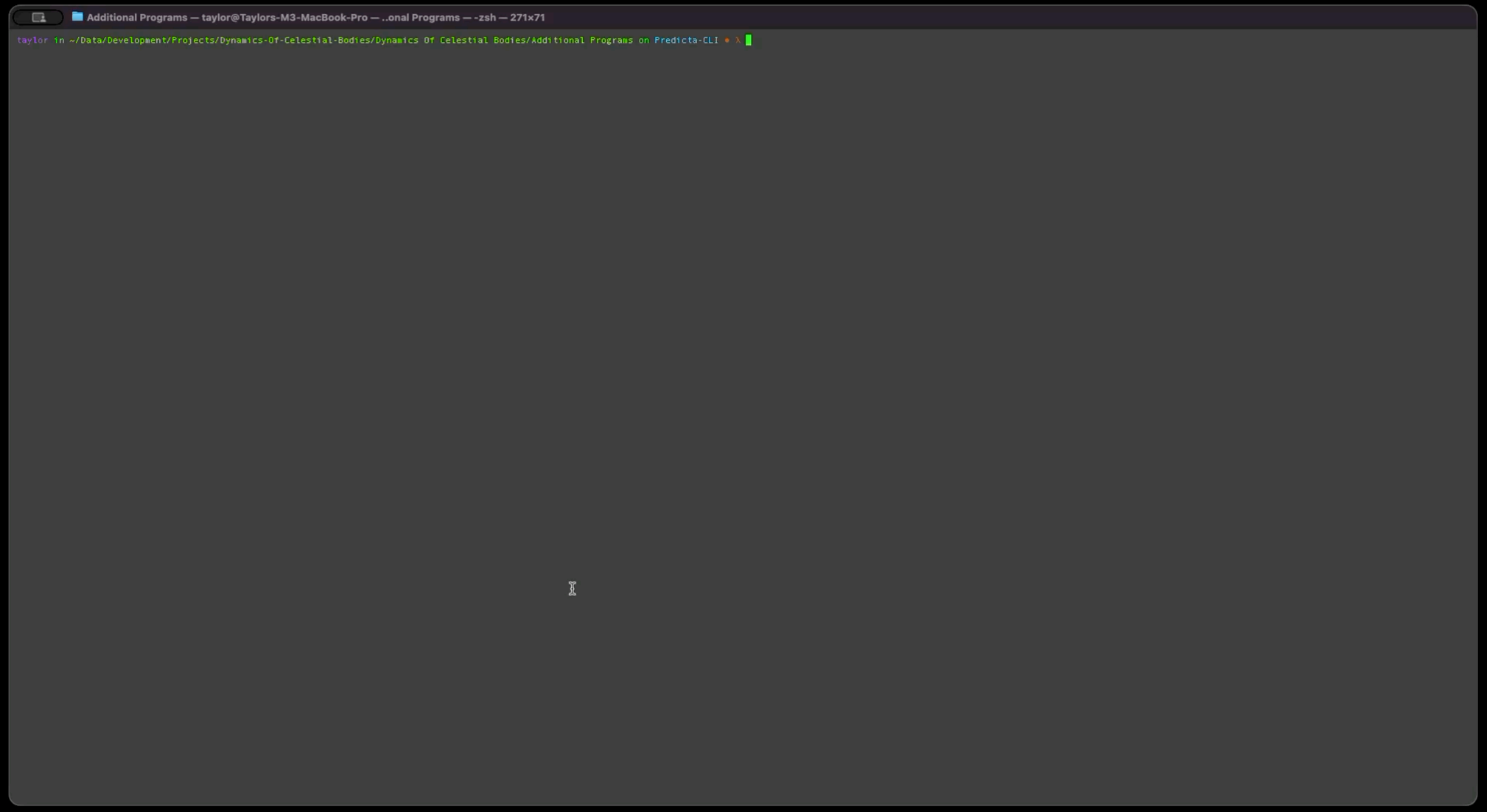
Algorithms Of Each Language

```python
def count_to(n: int, progress_every: int = 50_000_000) -> float:
    start = time.perf_counter()
    i = 0
    while i < n:
        i += 1
        if progress_every and (i % progress_every == 0):
            elapsed = time.perf_counter() - start
            print(f"Reached {i:,} / {n:,}  (elapsed: {elapsed:.2f}s)")
            sys.stdout.flush()
    end = time.perf_counter()
    return end - start
```

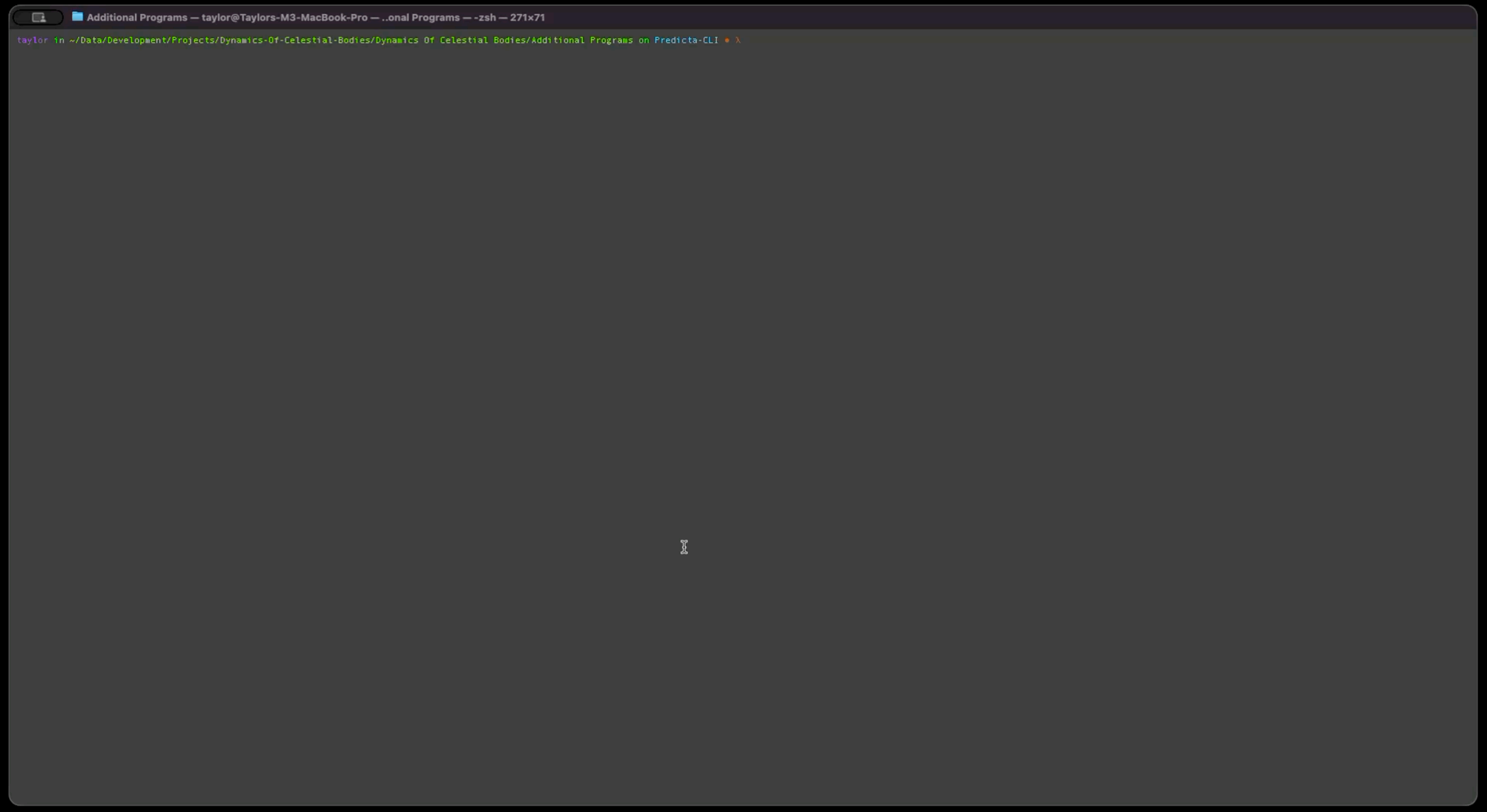
```cpp
double count_to(std::uint64_t n, std::uint64_t progress_every = 50'000'000ULL) {
    using clock = std::chrono::steady_clock;
    const auto start = clock::now();
    std::uint64_t i = 0;
    while (i < n) {
        ++i;
        if (progress_every && (i % progress_every == 0)) {
            const auto now = clock::now();
            const std::chrono::duration<double> elapsed = now - start;

            std::cout << "Reached " << i << " / " << n
                      << "  (elapsed: " << std::fixed << std::setprecision(2)
                      << elapsed.count() << "s)\n";
            std::cout.flush();
        }
    }
    const auto end = clock::now();
    const std::chrono::duration<double> elapsed = end - start;
    return elapsed.count();
}
```

Python


Additional Programs — taylor@Taylors-M3-MacBook-Pro — ..onal Programs — -zsh — 271×71
taylor in ~/Data/Development/Projects/Dynamics-Of-Celestial-Bodies/Dynamics Of Celestial Bodies/Additional Programs on Predicta-CLI

C++


Additional Programs — taylor@Taylors-M3-MacBook-Pro — ..onal Programs — -zsh — 271×71
taylor in ~/Data/Development/Projects/Dynamics-Of-Celestial-Bodies/Dynamics Of Celestial Bodies/Additional Programs on Predicta-CLI ● λ

Results

Python

Done. Elapsed time: 46.27 seconds

C++

Done. Elapsed time: 1.18 seconds

C++ is roughly 39.2 times faster at the same task

The Math

How Numerical Methods Work

The TLDR

- **What are they?** Numerical methods are algorithms that approximate solutions to problems that are too hard (or impossible) to solve exactly with algebra/calculus.
- **Core idea:** Replace a continuous problem with a sequence of small, manageable steps that a computer can compute.
- **How do they work?** Start from known information (initial conditions), apply a repeatable update rule, and accumulate results until the desired endpoint is reached.
- **Step size:** Smaller time steps usually give better accuracy, but require more computation; larger steps are faster but can miss important behavior of said system.
- **Trade-offs:** Methods differ in accuracy, stability, speed, and memory use, no single “best” method for every problem.
- **Validation:** Results are checked by testing on known cases, comparing multiple methods, and verifying expected properties.

A Primitive Method: FEDS

The Algorithm

Given an Initial Value Problem (**IVP**)

$$\frac{dy}{dt} = f(t, y) \qquad y(t_0) = y_0$$

pick a step size h, $t_{n+1} = t_n + h$, and then iterate

$$y_{n+1} = y_n + h \cdot f(t_n, y_n)$$

and that’s it, and it sucks.

A More Robust Method: RK4

The Algorithm

Given an Initial Value Problem (**IVP**)

$$\frac{dy}{dt} = f(t, y) \qquad y(t_0) = y_0$$

pick a step size h, $t_{n+1} = t_n + h$, and then compute what is called the slopes:

$$k_1 = f(t_n, y_n)$$
$$k_2 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2} \cdot k_1\right)$$
$$k_3 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2} \cdot k_2\right)$$
$$k_4 = f(t_n + h, y_n + h \cdot k_3)$$

Then update:

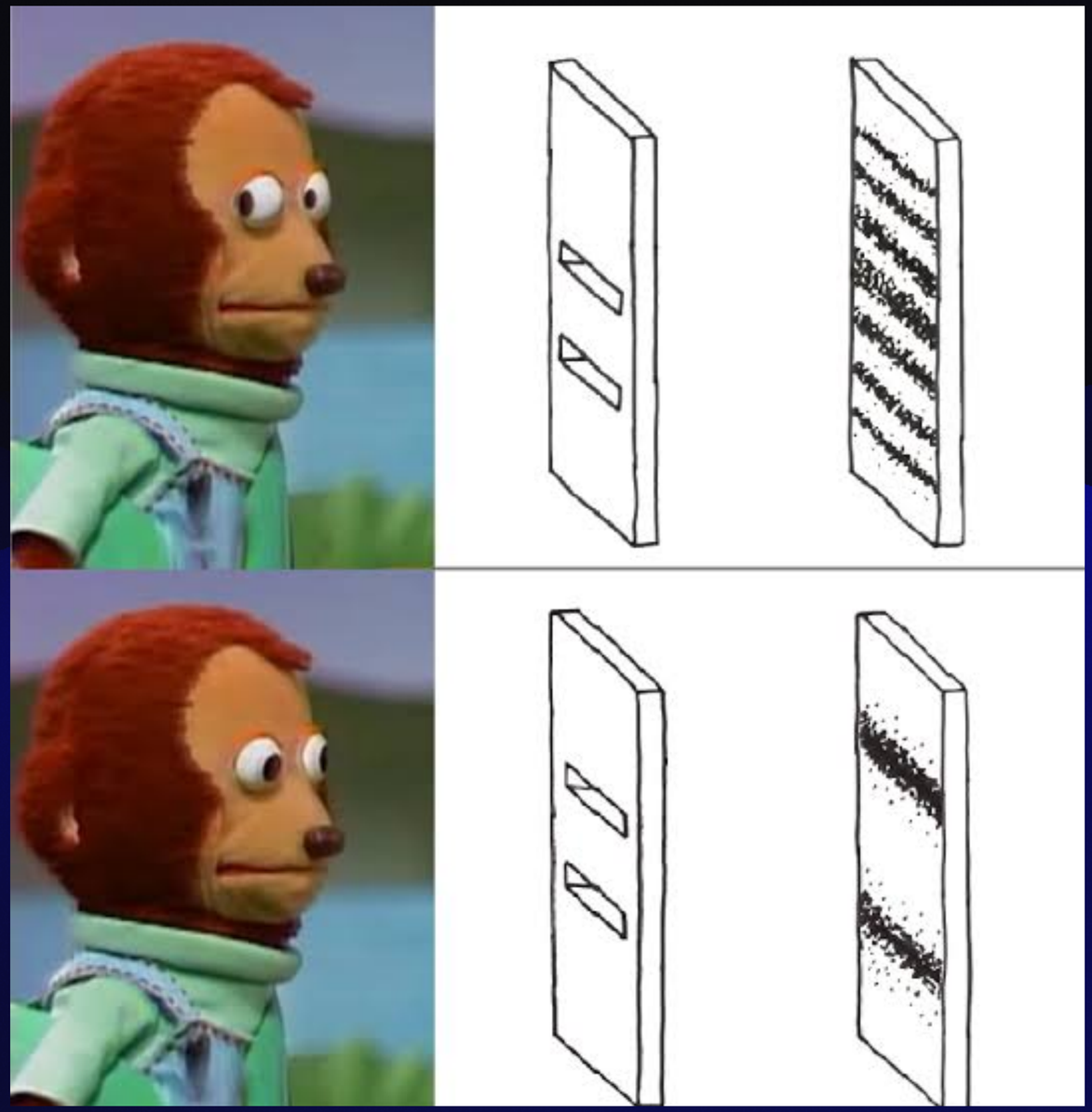
$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2 \cdot k_2 + 2 \cdot k_3 + k_4)$$

And it’s, pretty good.

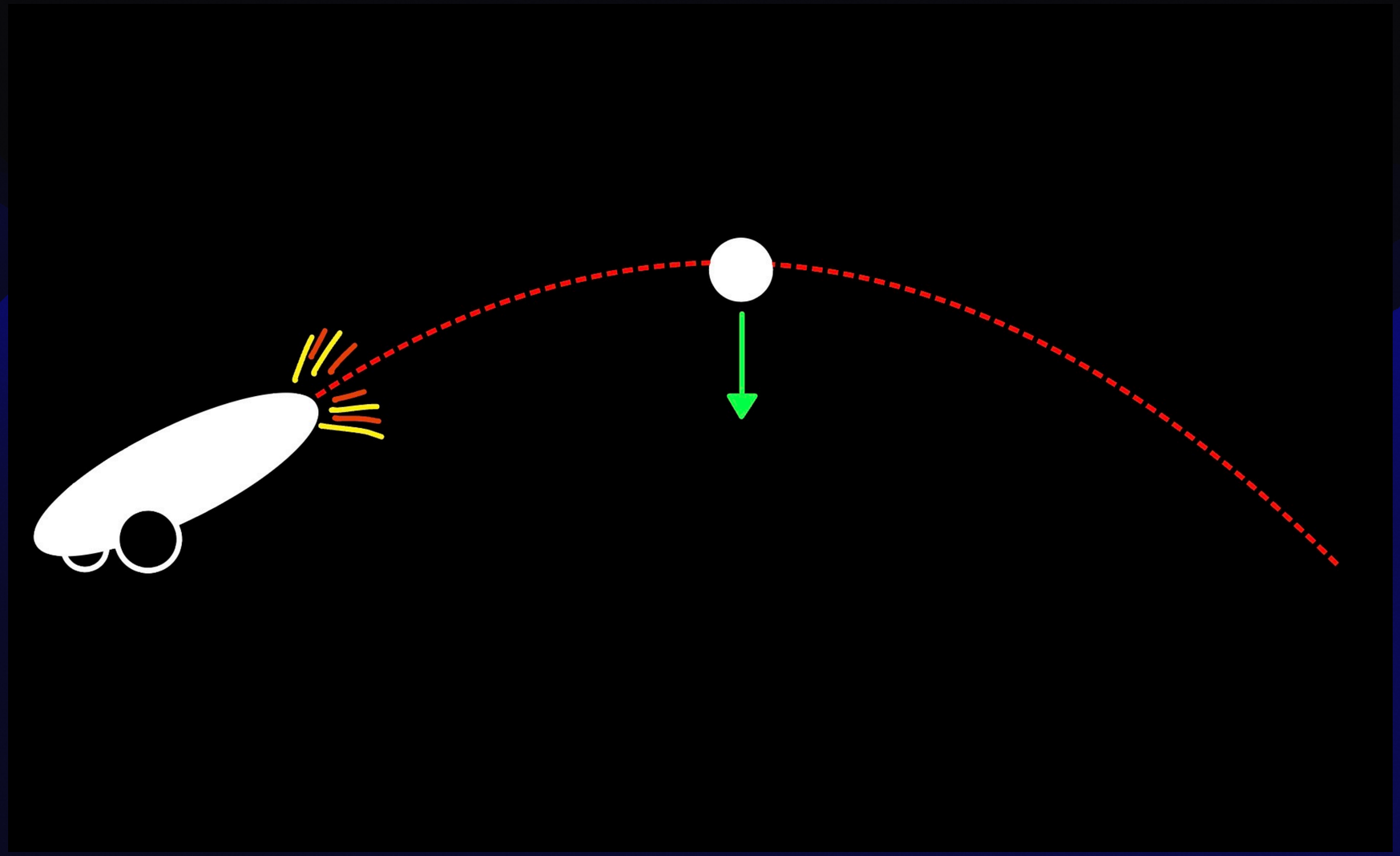
The Physics

The Physical Systems

- Single Body System
- Two Body System
- Three Body System
- N-Body System



Single Body System (Projectile Motion)



Single Body System (Projectile Motion)

Starting With Newtonian Gravitation

$$F = -\frac{GMm}{r^2}$$

Using Newton's Second Law

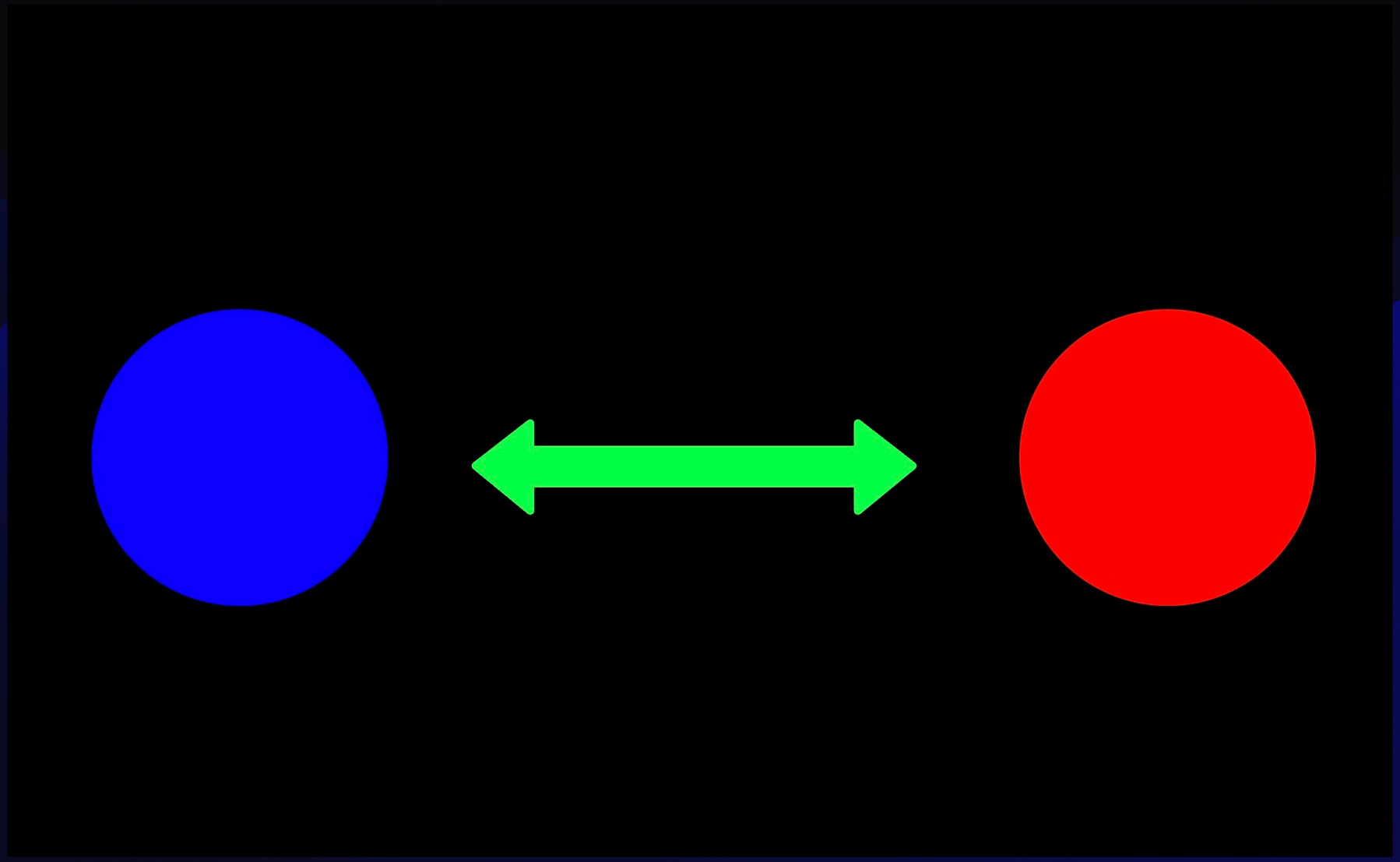
$$F = ma$$

Solving For Acceleration

$$a = -\frac{GM}{r^2}$$

$r = R + y$ where R is the radius of our body and y is the object's initial vertical position.

Two Body System



Two Body System

Starting With Newtonian Gravitation

$$\vec{F}_i = \frac{G m_i m_j}{r_{ij}^2} \vec{n}_{ij}$$

Using Newton's Second Law

$$\vec{F}_i = m_i \vec{a}_i$$

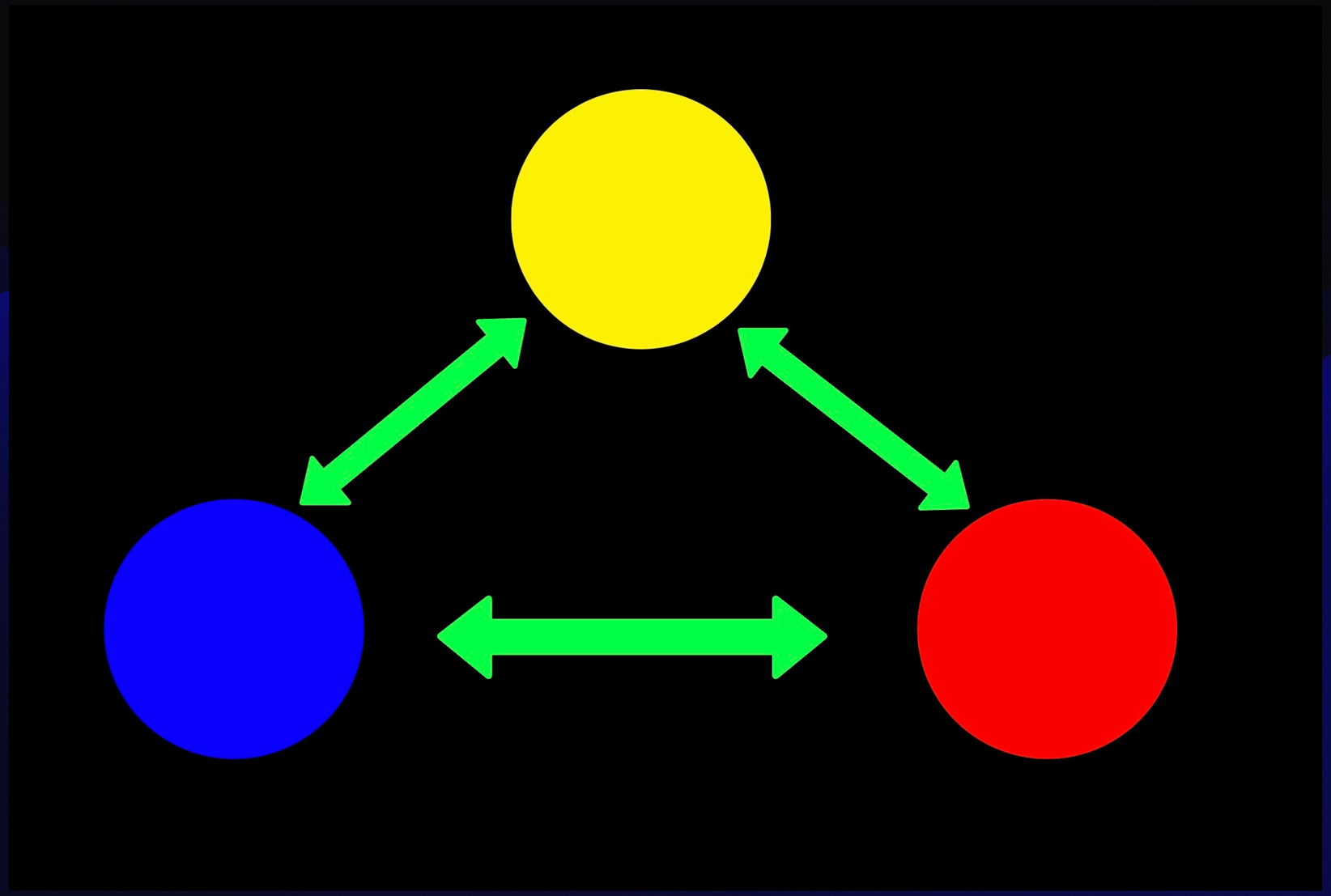
Extrapolating For Each Body

$$\vec{a}_i = \frac{G m_j}{r_{ij}^2} \vec{n}_{ij}$$

Where For Each Body

$$i, j \in \{1,2\}, \quad i \neq j \qquad \vec{r}_{ij} = \vec{r}_j - \vec{r}_i \qquad r_{ij} = \|\vec{r}_{ij}\| \qquad \vec{n}_{ij} = \frac{\vec{r}_{ij}}{r_{ij}}$$

Three Body System



Three Body System

Starting With Newtonian Gravitation

$$\overrightarrow{F}_i = \frac{Gm_i m_j}{r_{ij}^2}\vec{n}_{ij} + \frac{Gm_i m_k}{r_{ik}^2}\vec{n}_{ik}$$

Using Newton's Second Law

$$\overrightarrow{F}_i = m_i \vec{a}_i$$

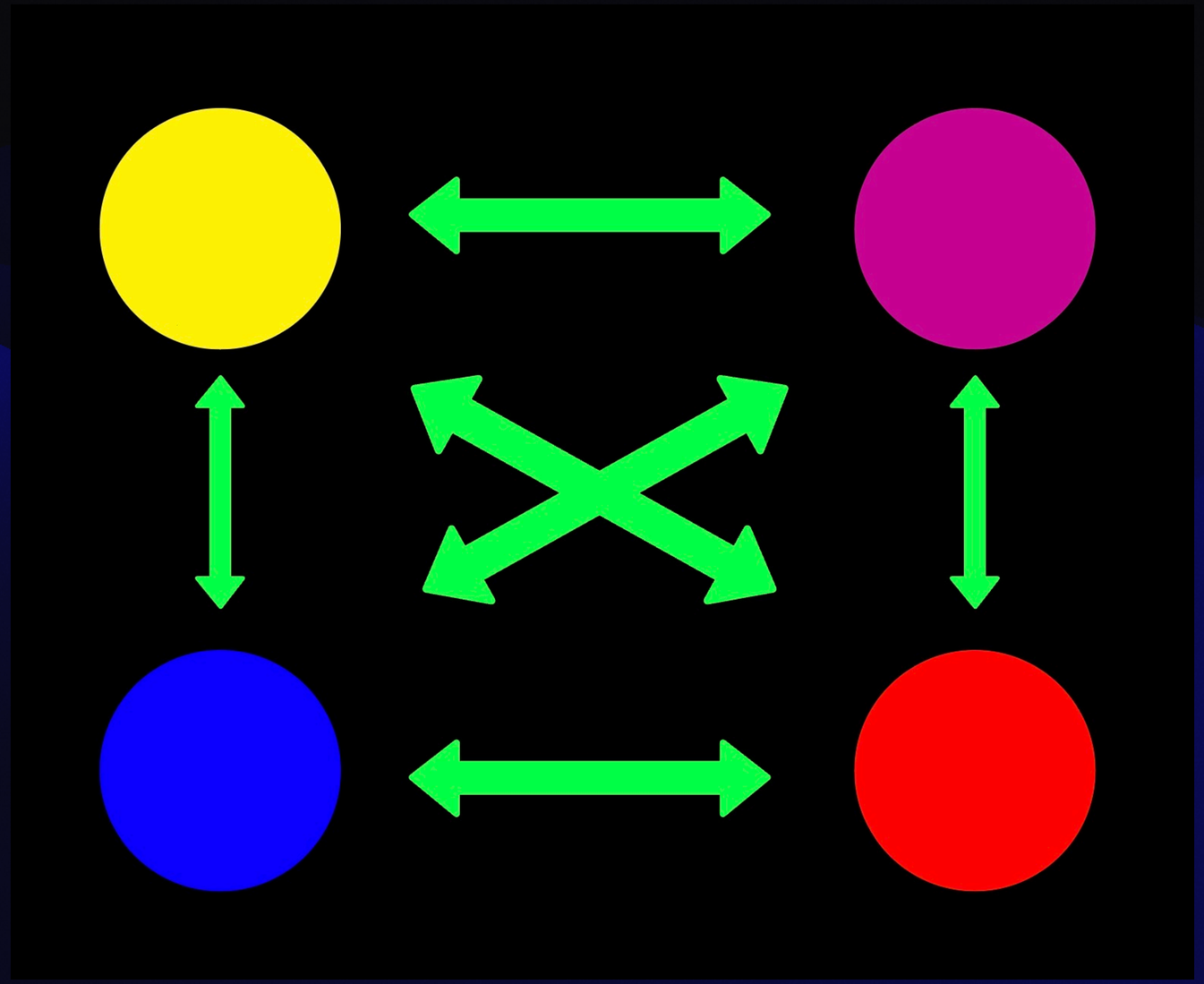
Extrapolating For Each Body

$$\vec{a}_i = \frac{Gm_j}{r_{ij}^2}\vec{n}_{ij} + \frac{Gm_k}{r_{ik}^2}\vec{n}_{ik}$$

Where For Each Body

$$i, j \in \{1,2,3\}, \quad i \neq j \qquad \vec{r}_{ij} = \vec{r}_j - \vec{r}_i \qquad r_{ij} = \|\vec{r}_{ij}\| \qquad \vec{n}_{ij} = \frac{\vec{r}_{ij}}{r_{ij}}$$

N-Body System



N-Body System

Starting With Newtonian Gravitation

$$\vec{F}_i = \sum_{\substack{j=1 \\ j \neq i}}^{N} \frac{G\, m_i\, m_j}{r_{ij}^2}\, \vec{n}_{ij}$$

Using Newton's Second Law

$$\vec{F}_i = m_i \vec{a}_i$$

Extrapolating For Each Body

$$\vec{a}_i = \sum_{\substack{j=1 \\ j \neq i}}^{N} \frac{G\, m_j}{r_{ij}^2}\, \vec{n}_{ij}$$

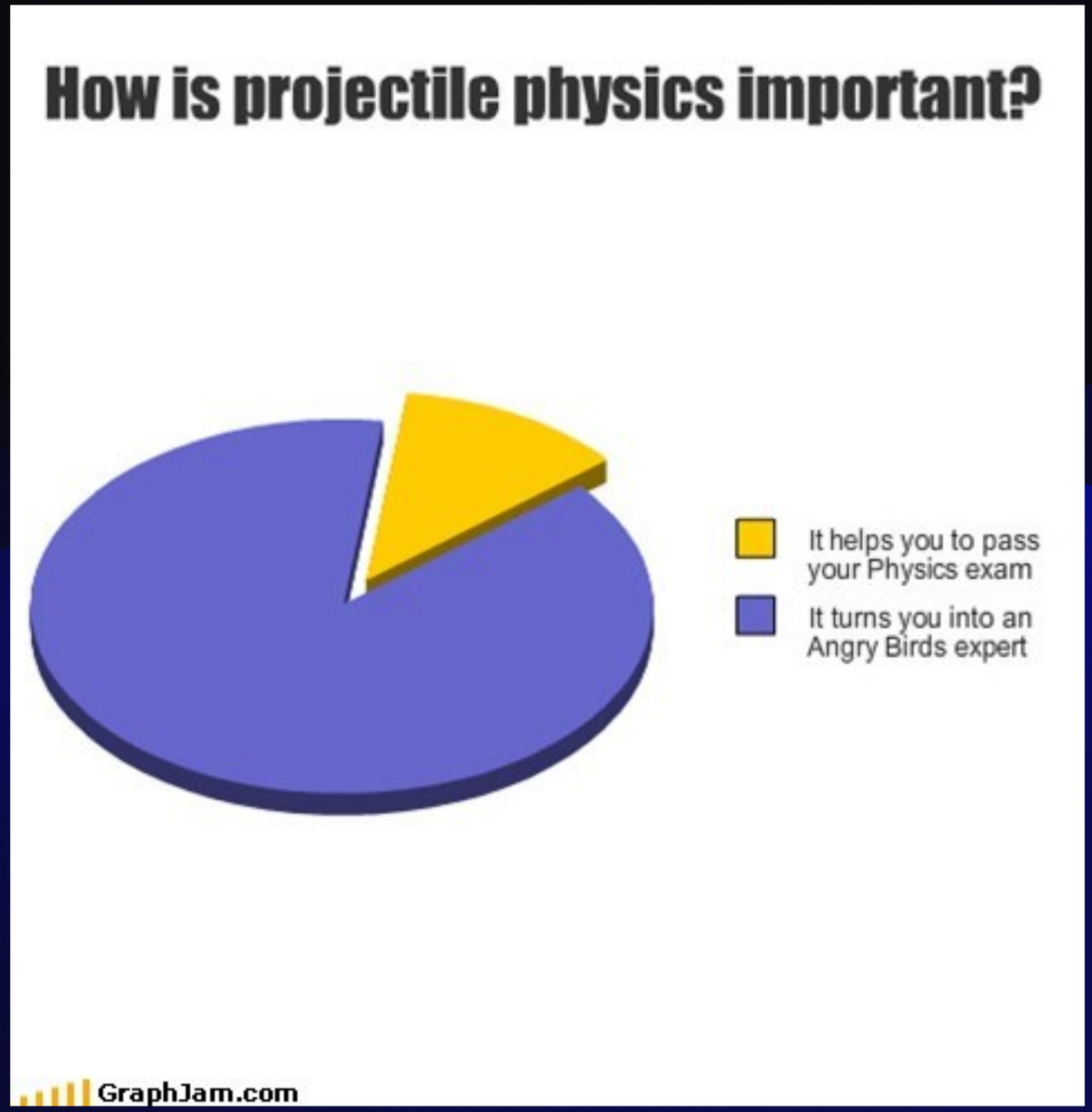
Where For Each Body

$$i, j \in \{1,2,\ldots,N\}, \quad i \neq j, \qquad \vec{r}_{ij} = \vec{r}_j - \vec{r}_i, \qquad r_{ij} = \|\vec{r}_{ij}\|, \qquad \vec{n}_{ij} = \frac{\vec{r}_{ij}}{r_{ij}}$$

Refactored Results

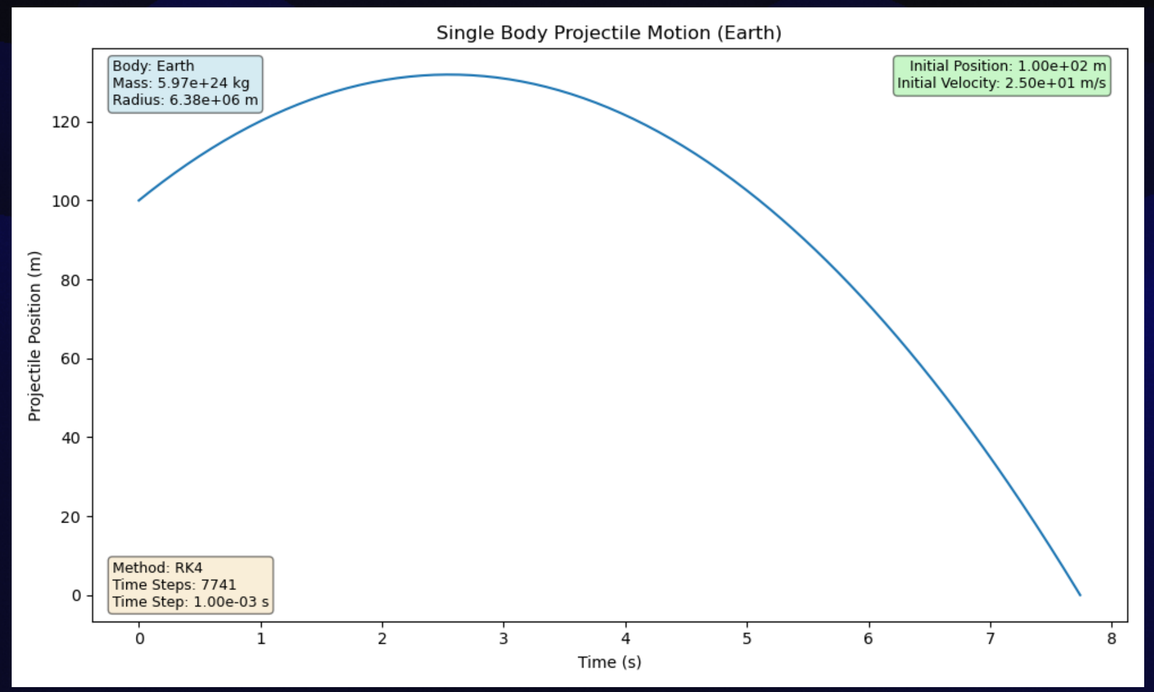
Single Body Baseline

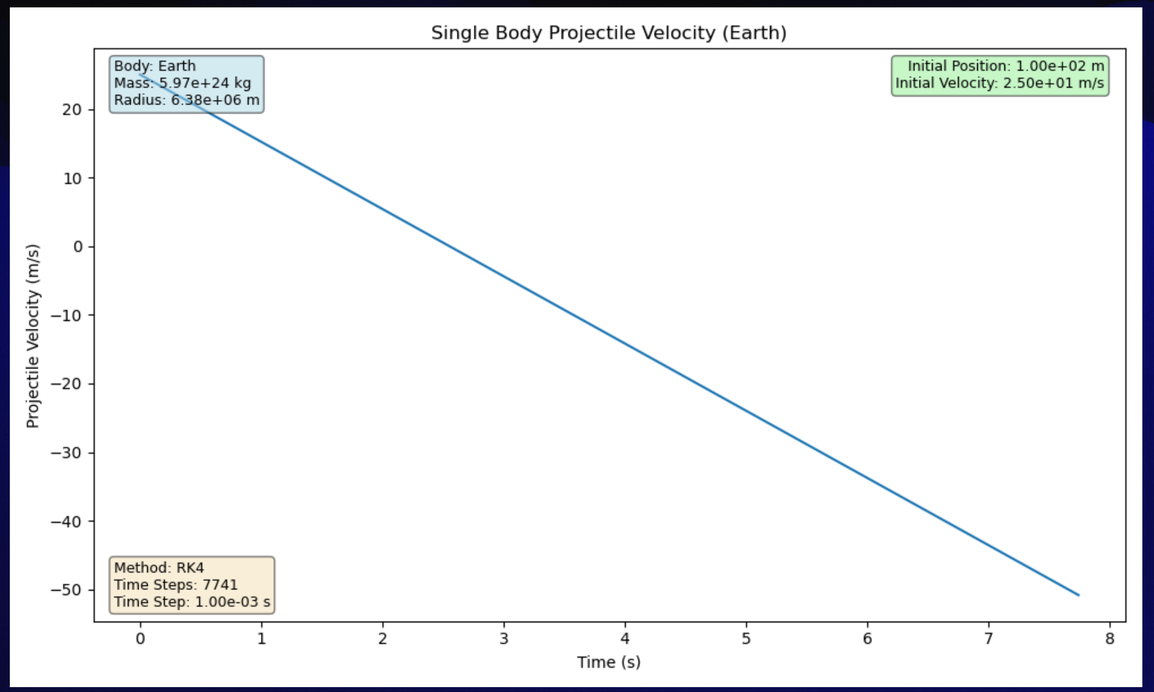
- We first test on Earth, and compare results with kinematics.
- Once verified, we can test on other planets, masses, etc.



Single Body System On Earth

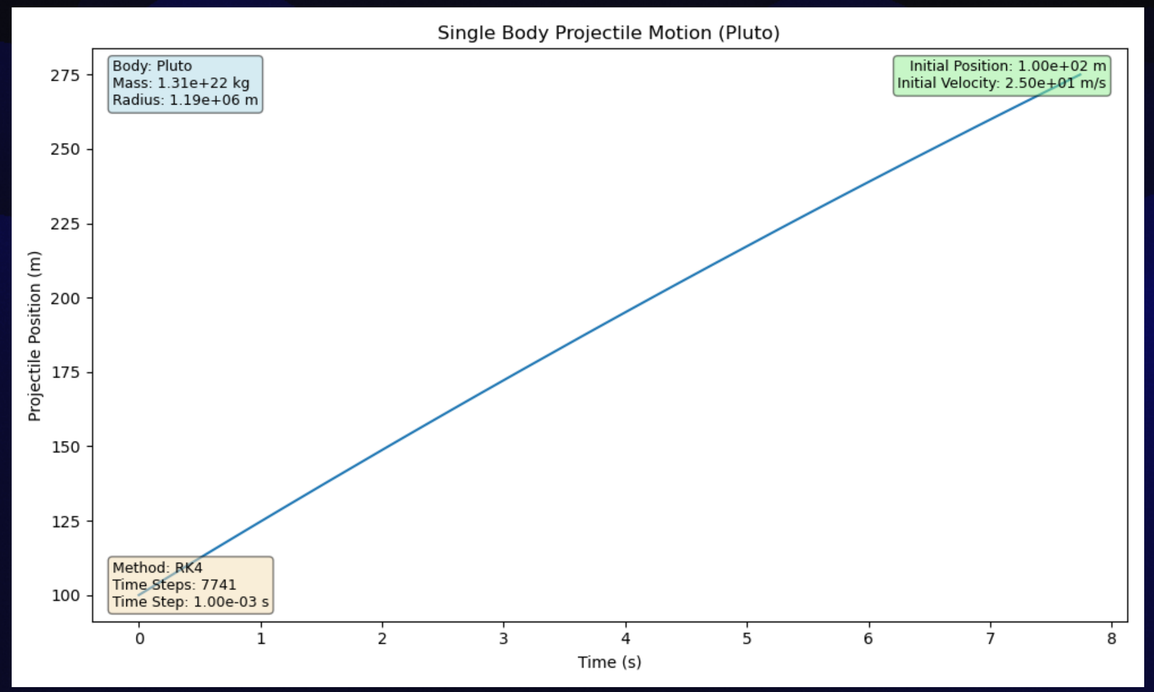
$$y_0 = 100\ (m), \qquad v_{y_0} = 25\ (m/s), \qquad t = 7.74\ (s), \qquad h = 0.001\ (s)$$

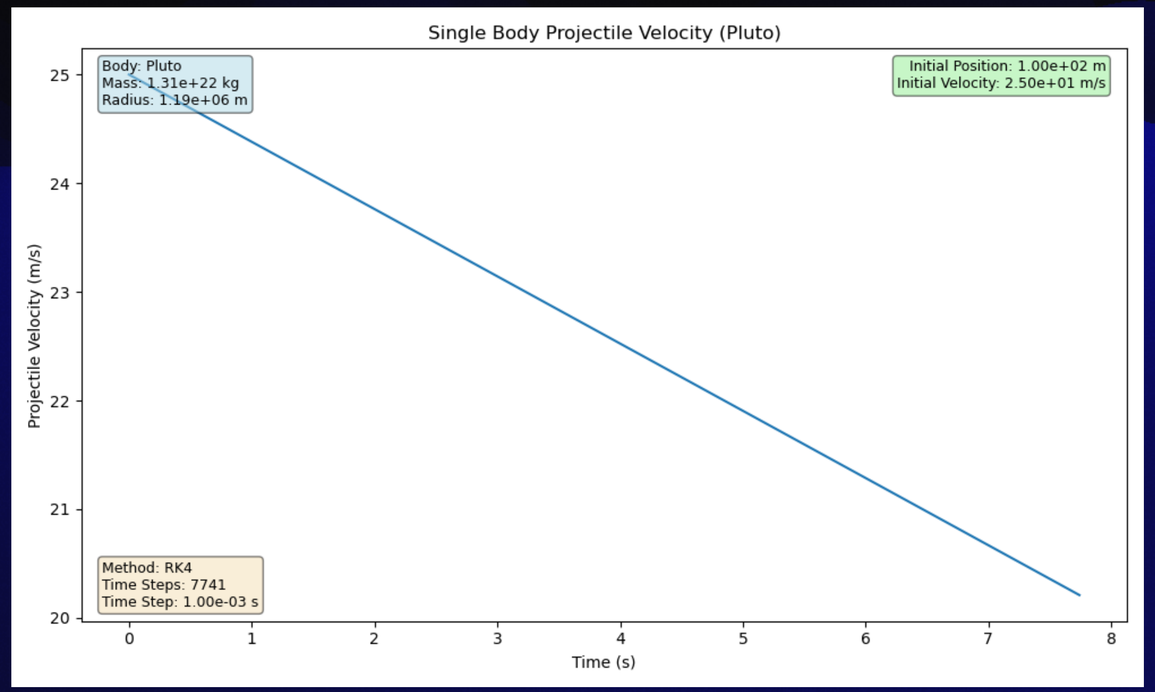




Single Body System On Pluto

$$y_0 = 100\ (m), \qquad v_{y_0} = 25\ (m/s), \qquad t = 7.74\ (s), \qquad h = 0.001\ (s)$$

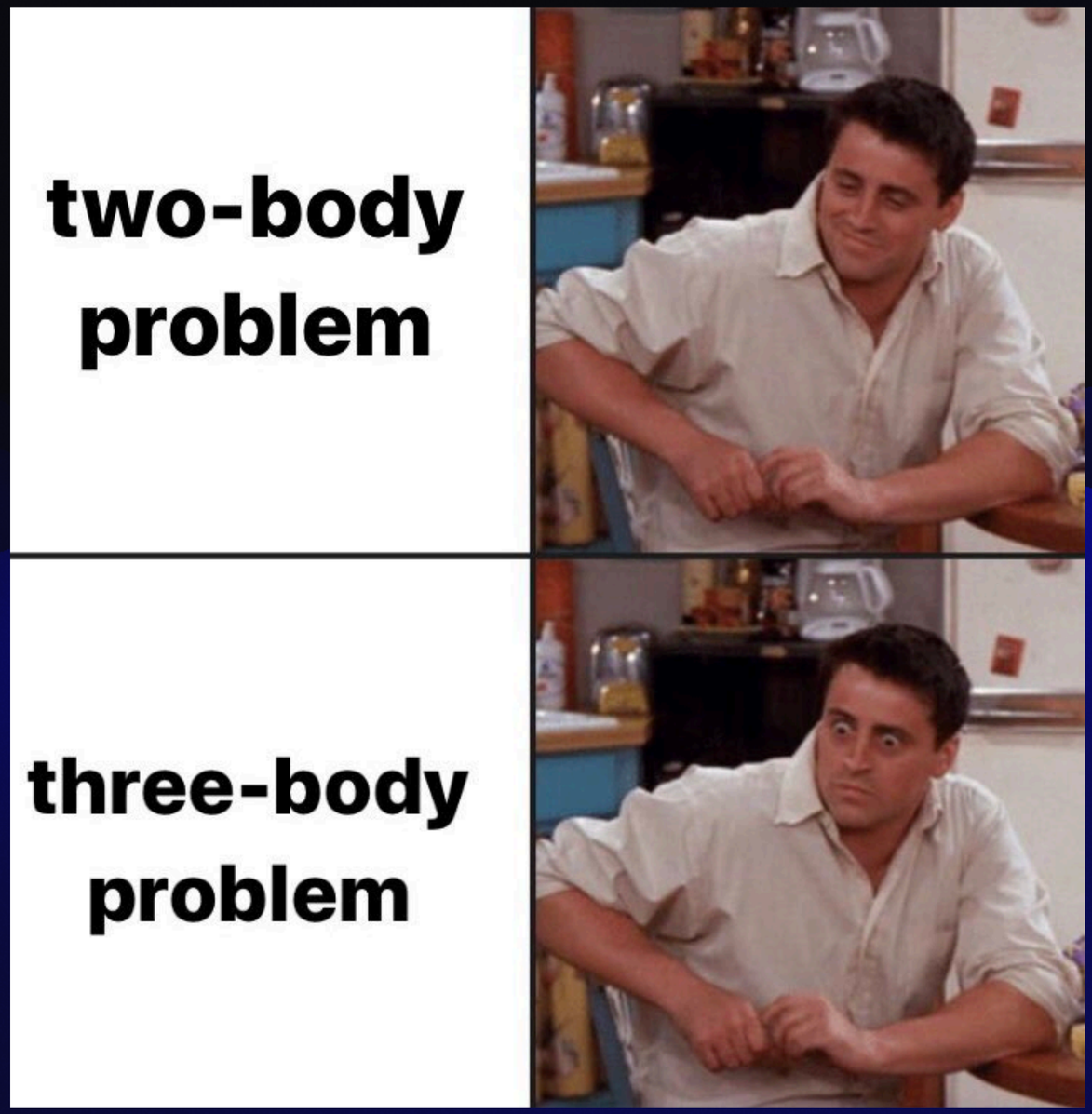




Two Body Baseline

Next Step In Complexity

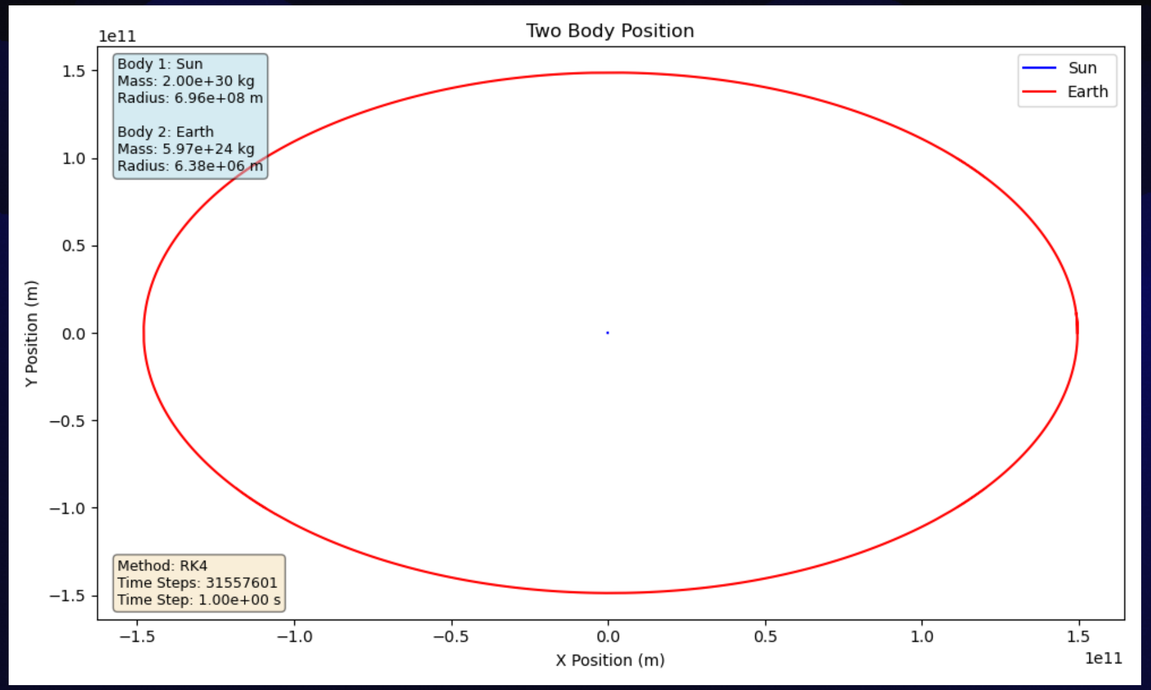
- We now test our numerical solvers with that of known systems.
- After the results are verified, we can test more unknown systems.
- And then move on to the three body problem.

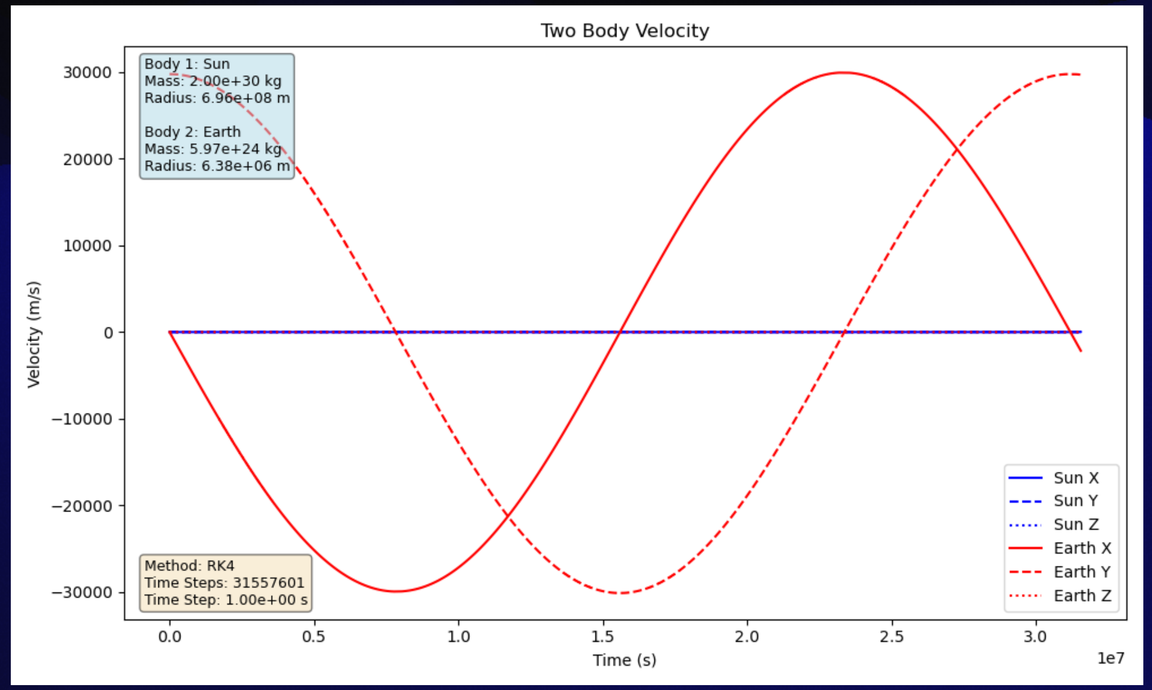


Two Body System Of Earth

One orbital period of Earth is plotted:

$$h = 1\ (s), \qquad t = 365.25\ (Earth\ Days), \qquad h_T = 31{,}557{,}601$$

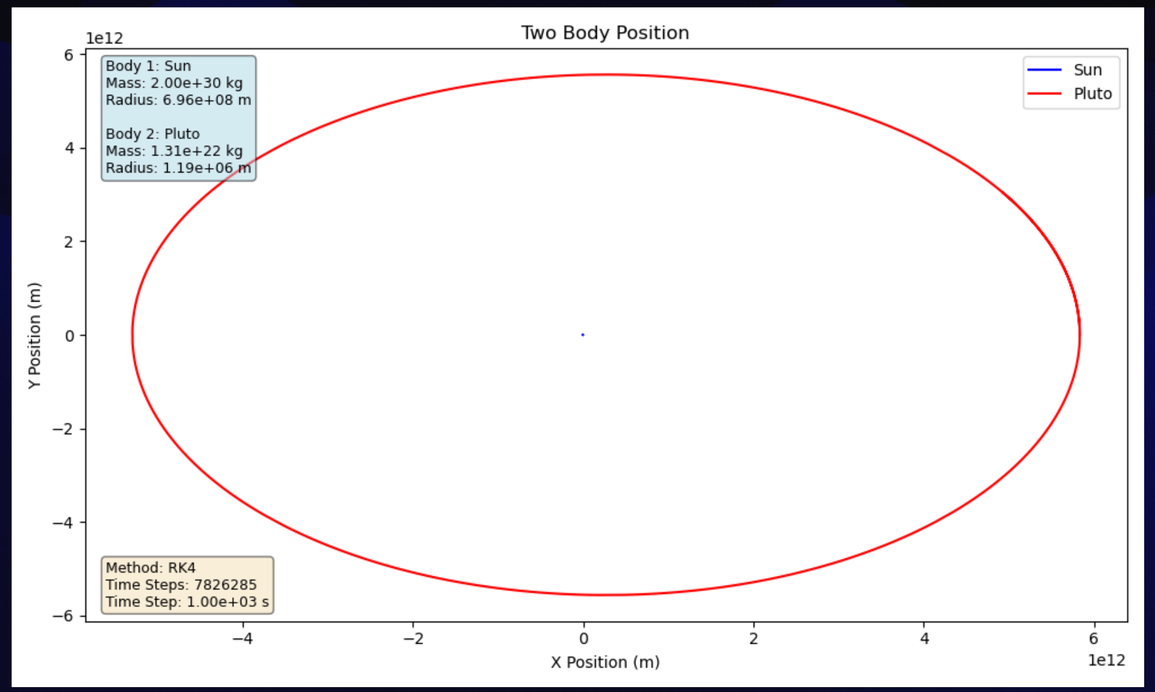


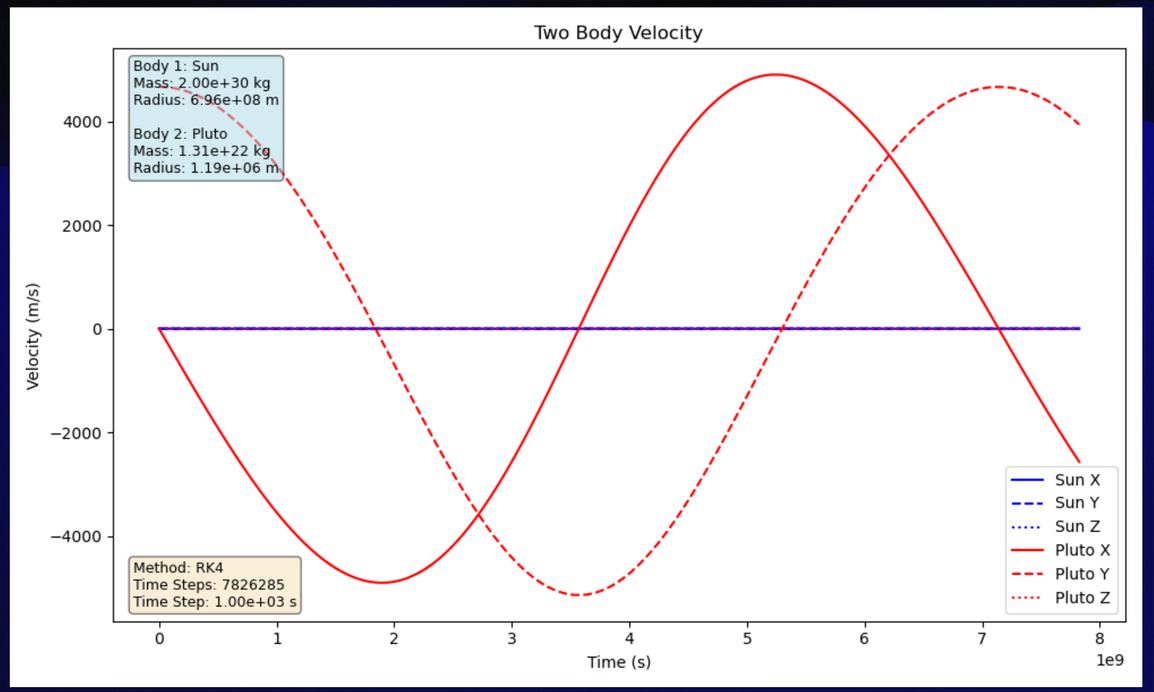


Two Body System Of Pluto

One orbital period of Pluto is plotted:

$$h = 1{,}000\ (s), \qquad t = 248\ (Earth\ Years), \qquad h_T = 7{,}826{,}285$$

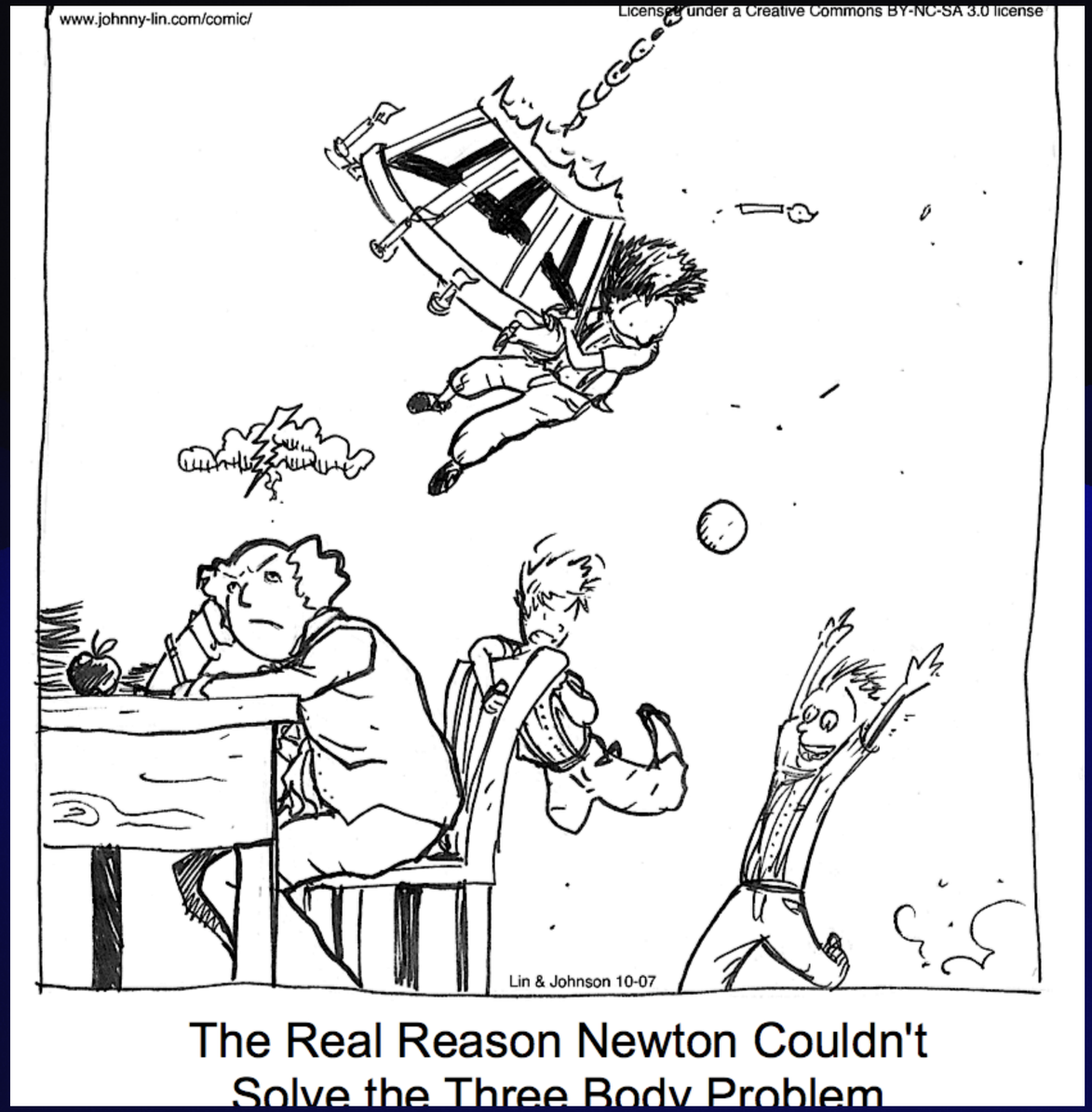




Three Body Baseline

Almost To N-Body

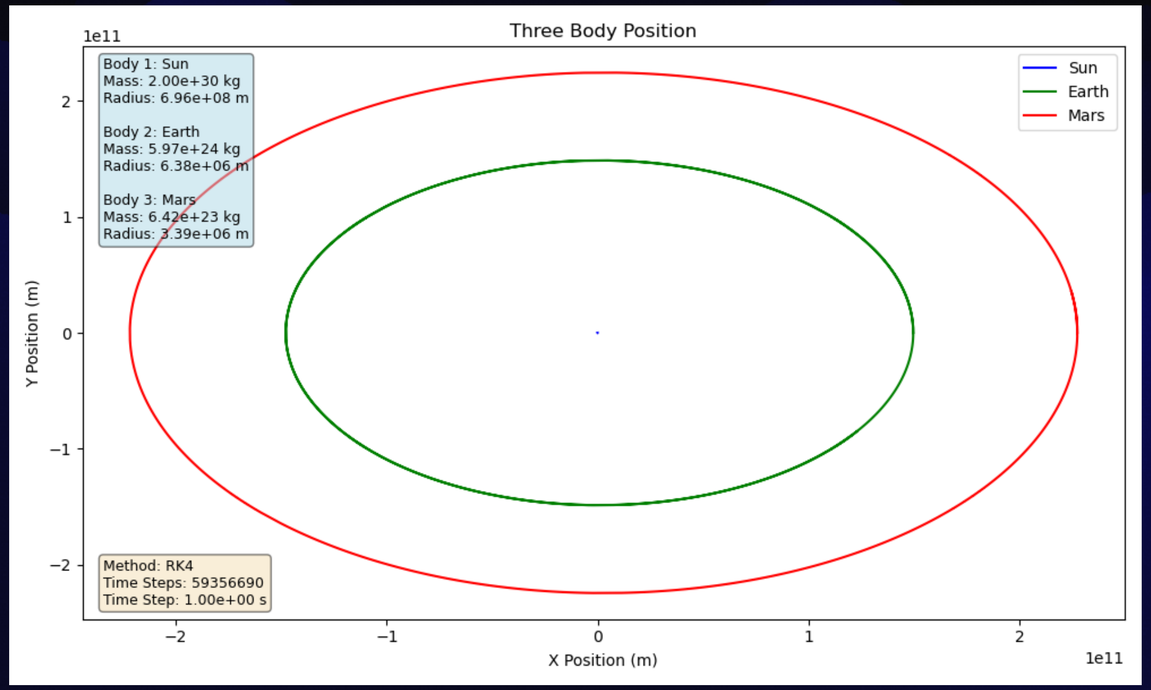
- Knowing that the Two Body solvers are working, we now can begin to move on to a more complicated system.
- We test with known orbits again, and then test with foreign systems.

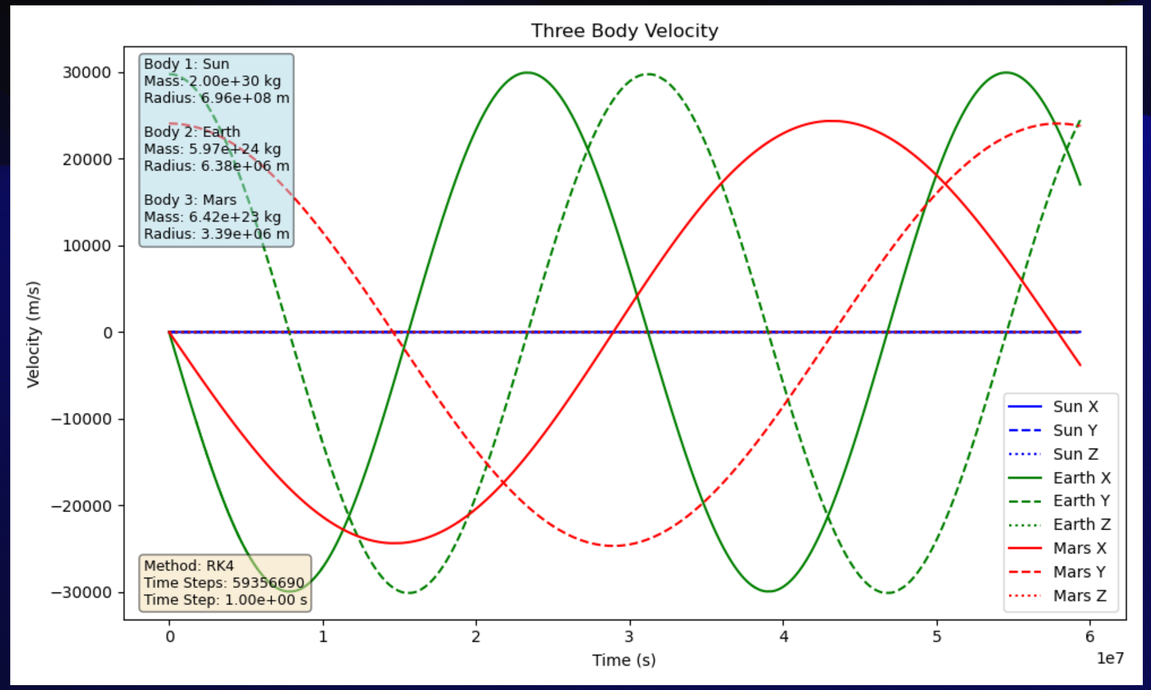


Three Body System - Sun, Earth, Mars

One orbital period of Mars Plotted:

$$h = 1\ (s), \qquad t = 687\ (Earth\ Days), \qquad h_T = 59{,}356{,}690$$

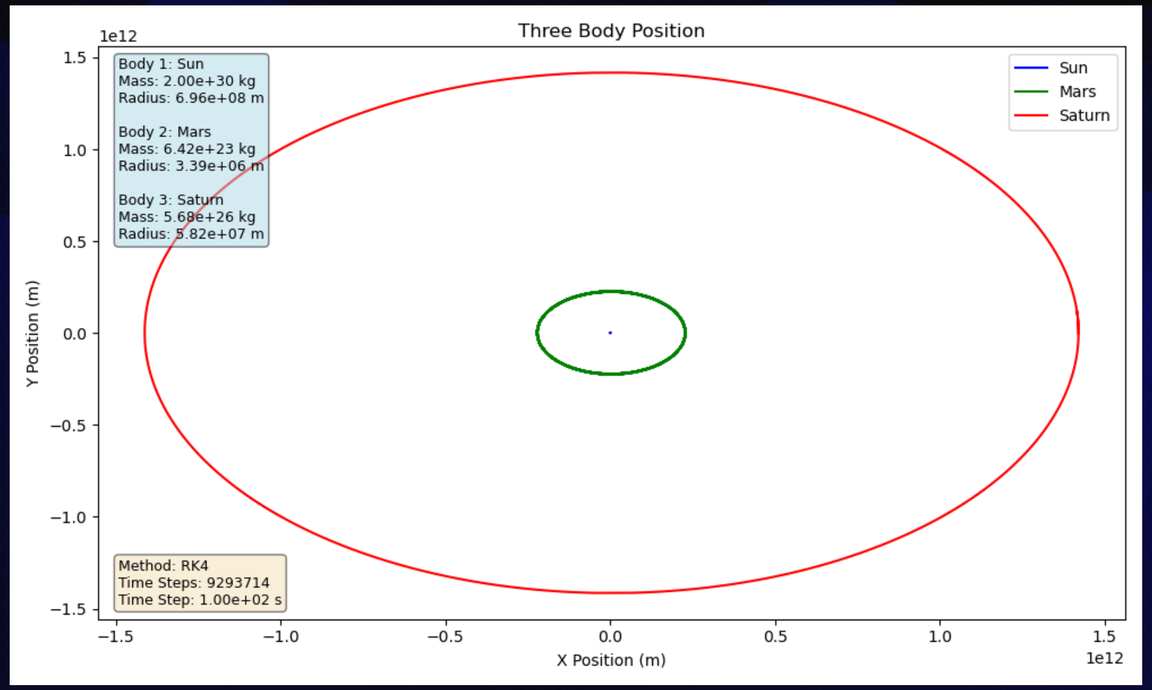


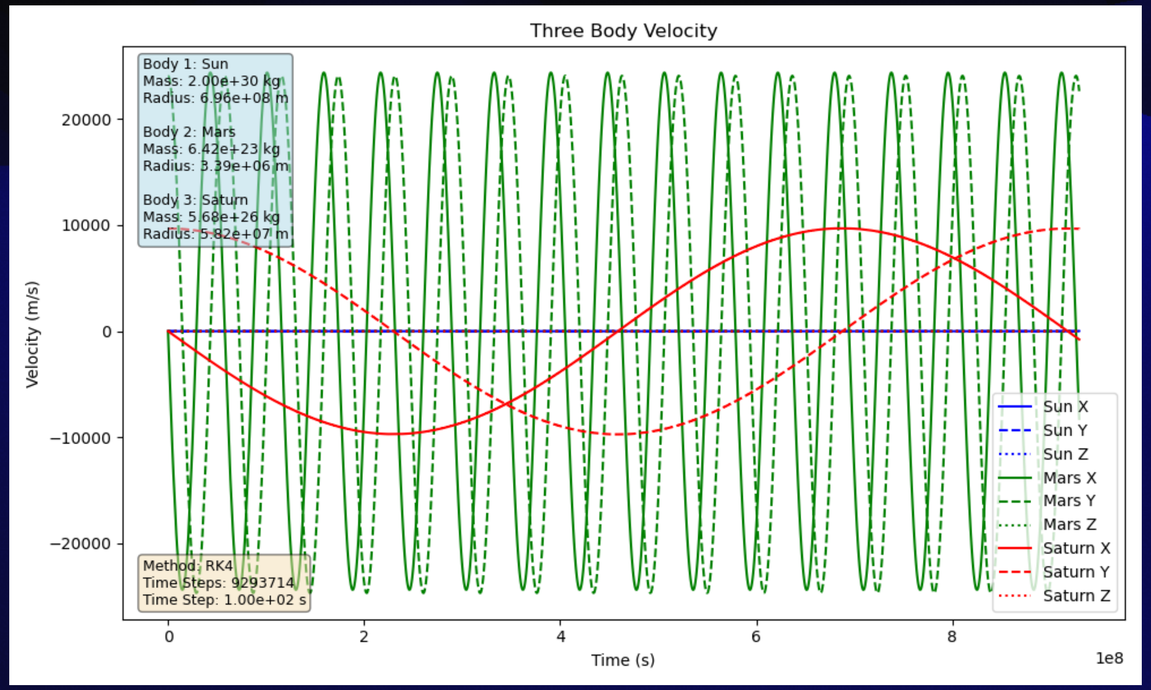


Three Body System - Sun, Mars, Saturn

One orbital period of Saturn Plotted:

$$h = 100\ (s), \qquad t = 29.45\ (Earth\ years), \qquad h_T = 9{,}151{,}705$$

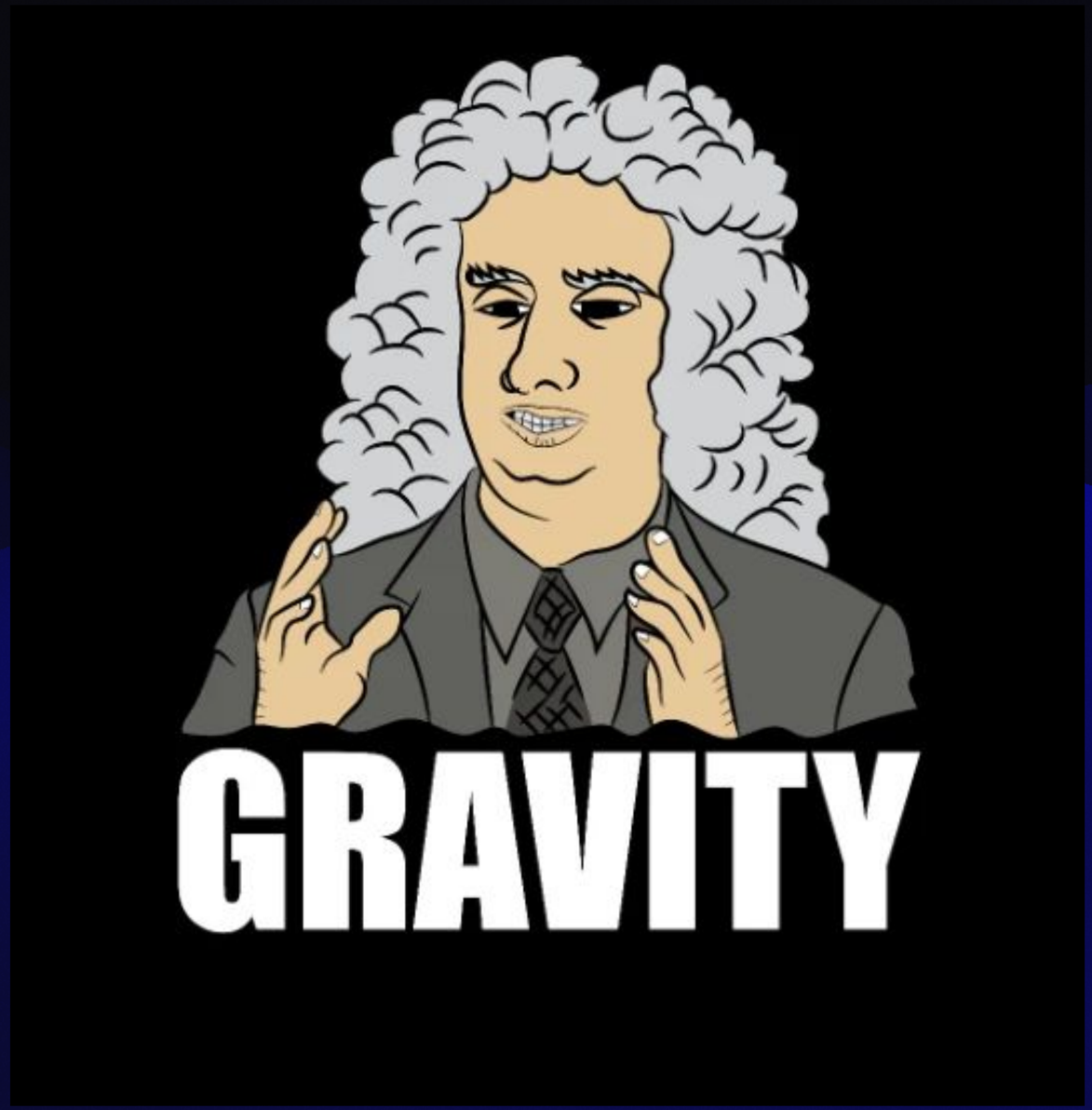




N-Body System

The Final Test

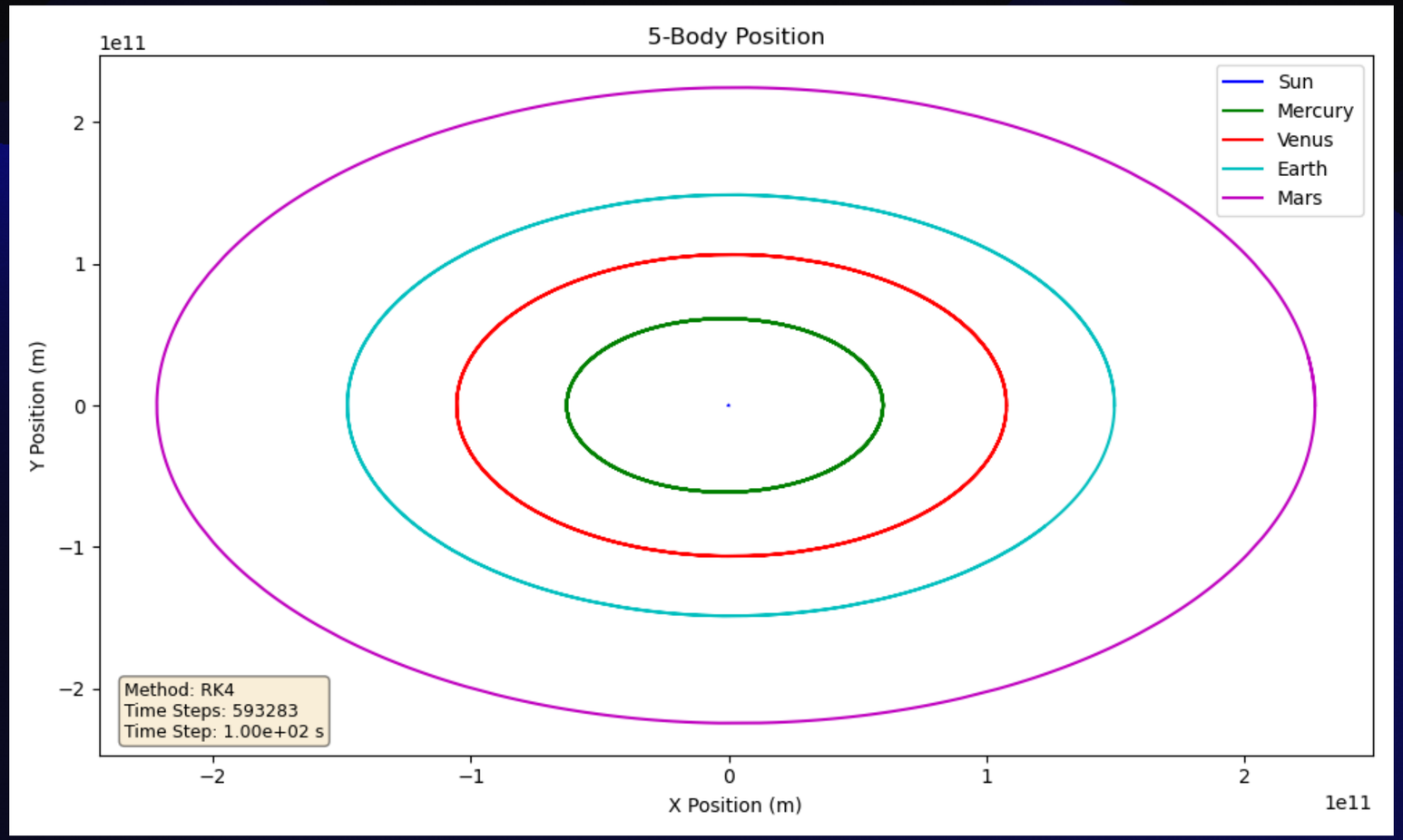
- Now that we have verifiable (kinda) solutions from the Three Body system, we can move on to the most complex system for this problem.
- Orbits of planets are plotted as well as random systems.



N-Body System - Sun To Mars

One orbital period of Mars is plotted along with all terrestrial planets:

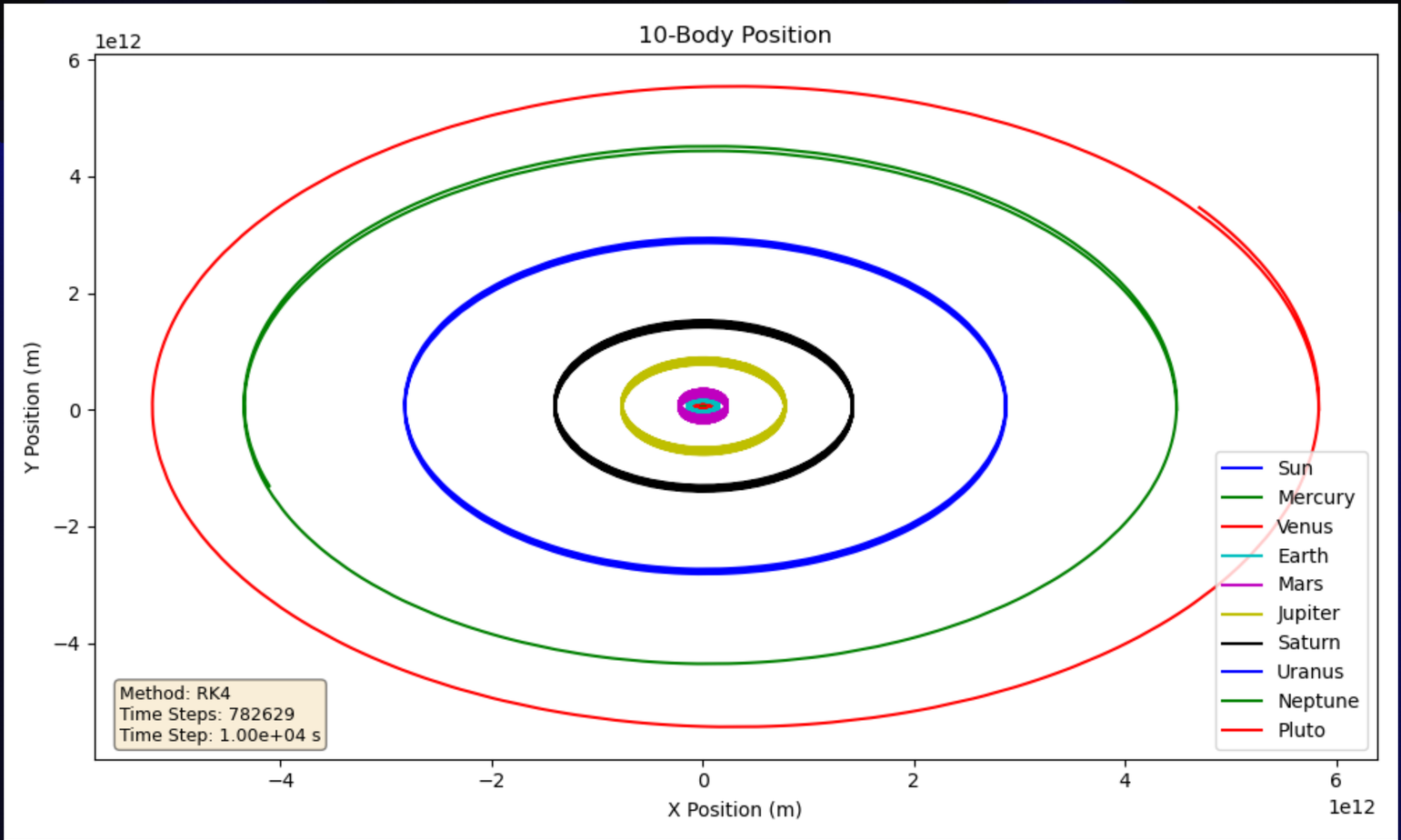
$$h = 100\ (s), \qquad t = 687\ (Earth\ Days), \qquad h_T = 593{,}283$$



N-Body System - Solar System

One orbital period of Pluto is plotted with the 8 planets:

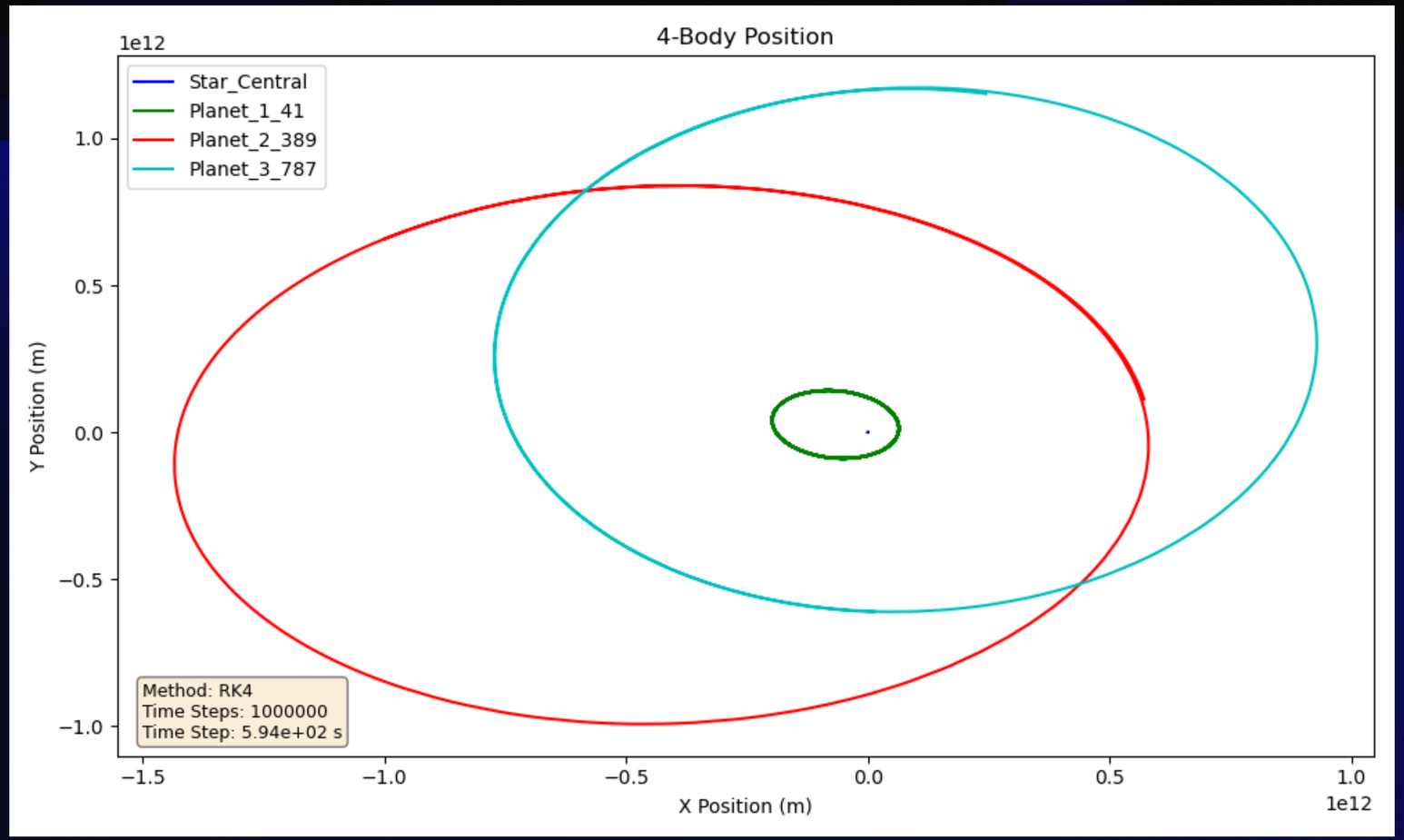
$$h = 10{,}000\ (s), \qquad t = 249\ (Earth\ Years), \qquad h_T = 782{,}629$$



N-Body System - Random System 1

Random 4 Body System plotted:

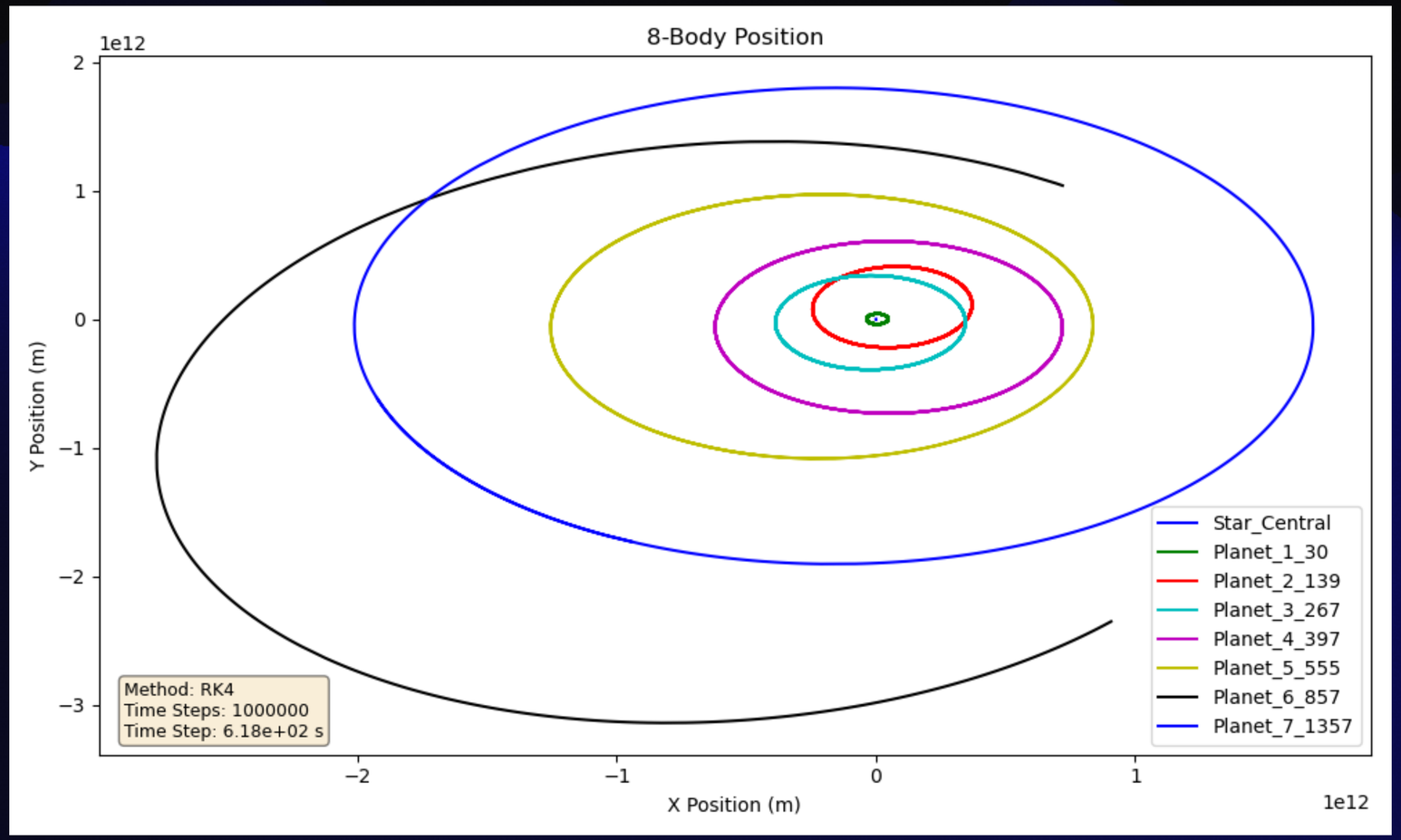
$$h = 594\ (s), \qquad t = 18.82\ (Earth\ Years), \qquad h_T = 1{,}000{,}000$$



N-Body System - Random System 2

Random 8 Body System plotted:

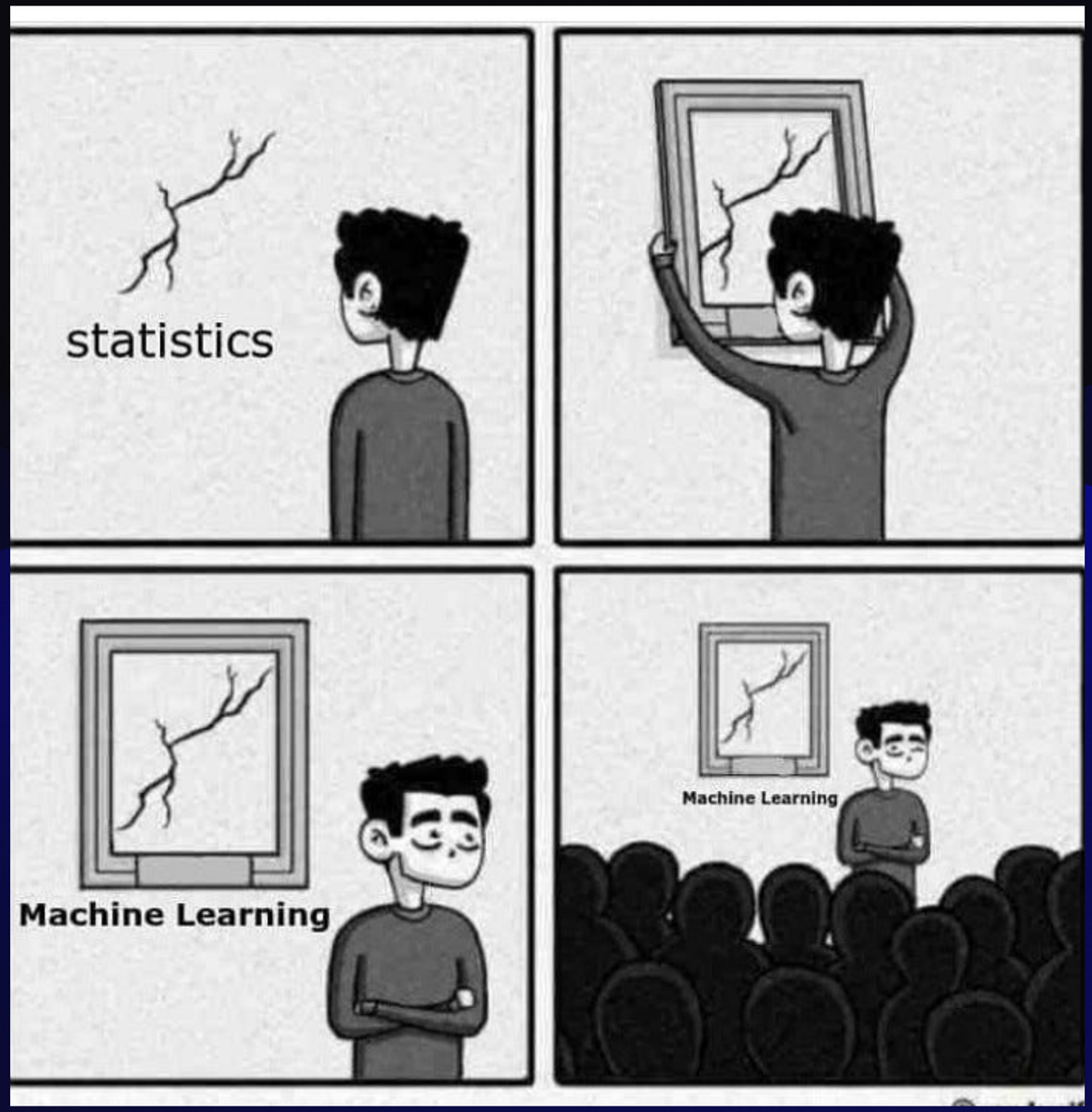
$$h = 618\ (s), \qquad t = 19.58\ (Earth\ Years), \qquad h_T = 1{,}000{,}000$$



Physics Informed Neural Network

Machine Learning
With A PINN

- PINNs as a whole
- How it was implemented
- The results



PINNs

At A High Level

- **What they are:** A neural network trained to satisfy known physics (usually involves differential equations).
- **What they do:** Typically learn a parameter from data, used to make inferences on system with learned parameter.
- **Why they are used:** Often useful when the physics for a given system is trusted but measurements or something similar in a model is missing effects.
- **Benefits in data:** They perform relatively well due to extra supervision in training, they can work with limited, noisier, or sparse data.
- **Challenges with them:** Data production techniques, training time and refinement, they struggle with discontinuities, chaotic systems, long time horizons, etc.

What The Model Does

Predicta

- **Goal:** Train a neural network to make the following prediction: Given the system “right now”, predict the **acceleration** at this moment.
- **Why acceleration:** In a Single Body (projectile motion) system, we can use kinematics to predict position and velocity from this learned acceleration.
- **PINN-ness:** Instead of trying to predict the entire trajectory from raw data, it predicts local dynamics with the learned acceleration of the system.
- **Main parameters fed:** The model primarily trains on the following:
  - Initial position and velocity
  - Initial time, time step, and total time
  - Body’s mass, radius, initial position, initial velocity

How It Was Implemented

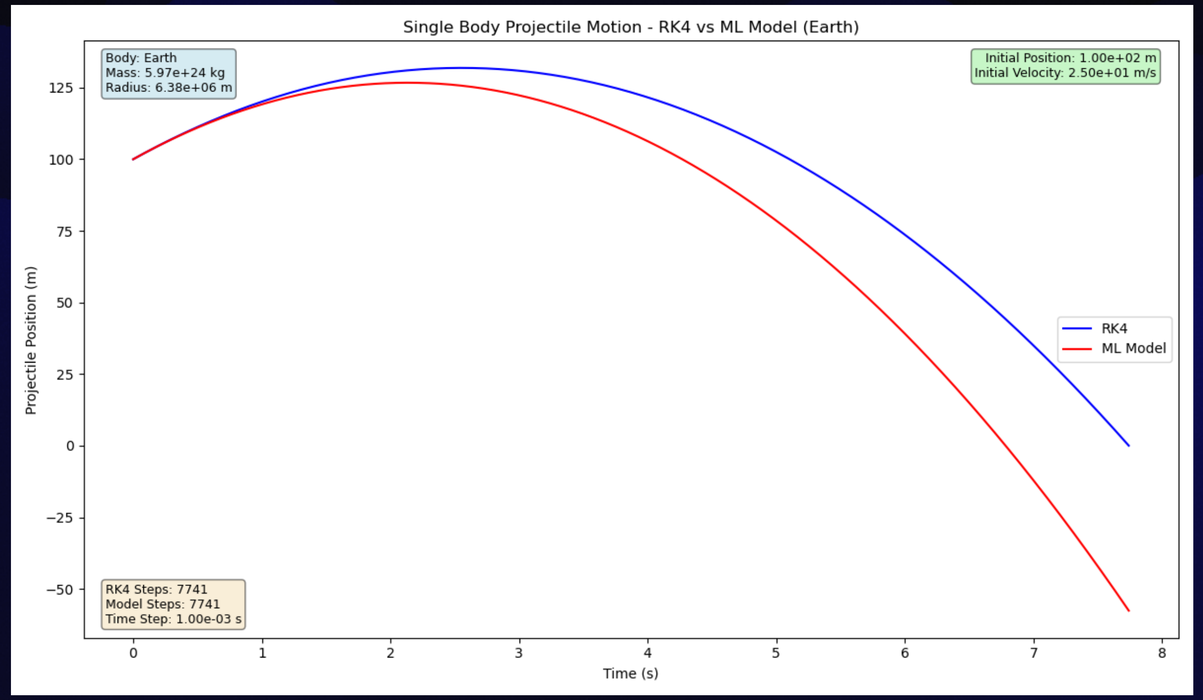
LibTorch MLP

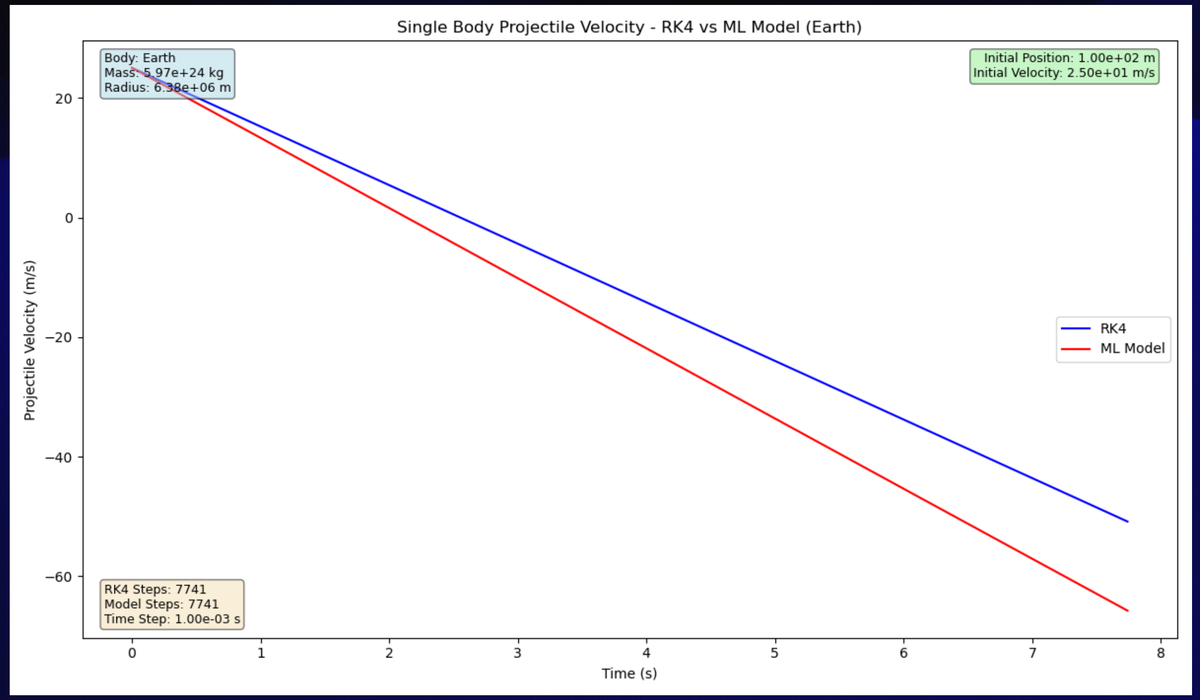
- **Language:** Entire data generation pipeline and machine learning model is written in C++.
- **Data generation:** Random initial conditions were fed to RK4 solver to generate.
  - The randomly generated data is specific to a body (e.g. Earth, Mars, Jupiter, etc.)
- **Body specific models:** From the body specific data, a body specific model is trained.
- **Training:** For each new model, 800 “data points” are created for training, 100 for validation, 100 for testing.
  - Resuming training does not create new validation or test data points, only 800 for training.
- **Each session:** One training session consists of 25 epochs:
  - Fresh models complete one epoch in about 60 seconds.
  - When resuming training, every 800 added training data points adds about 60 seconds to each epoch.

Prediction 1

Projectile launched from Earth, 1 training session

$$y_0 = 100\ (m), \qquad v_{y_0} = 25\ (m/s), \qquad t = 7.74\ (s), \qquad h = 0.001\ (s)$$

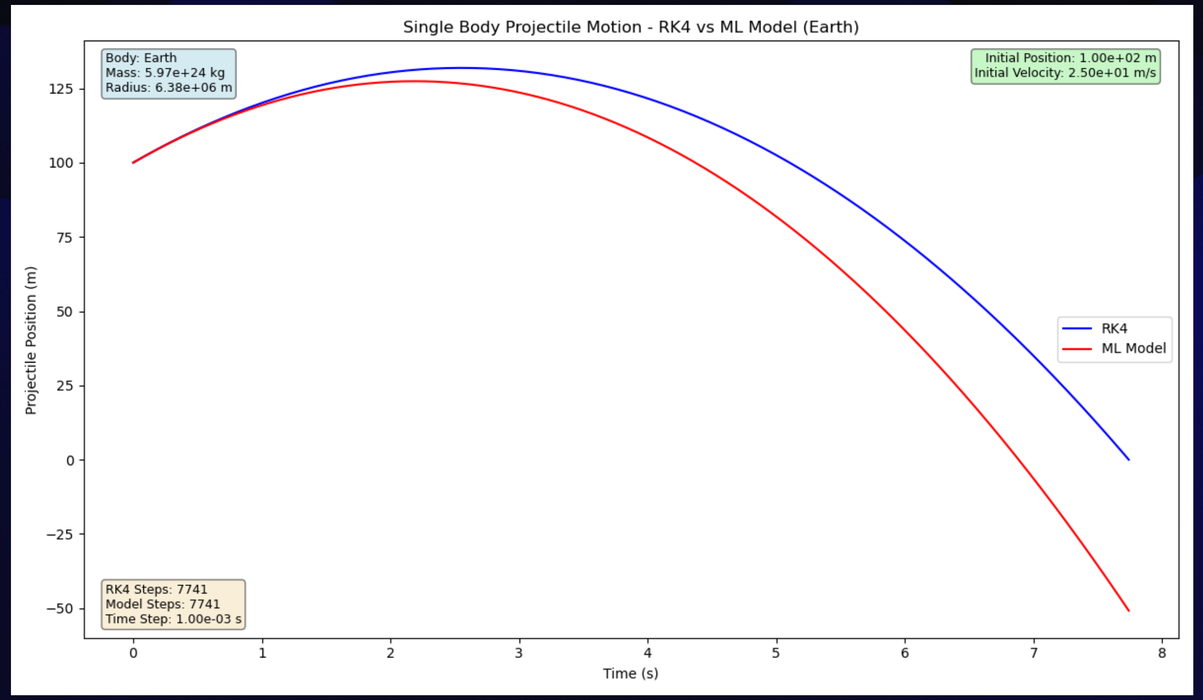


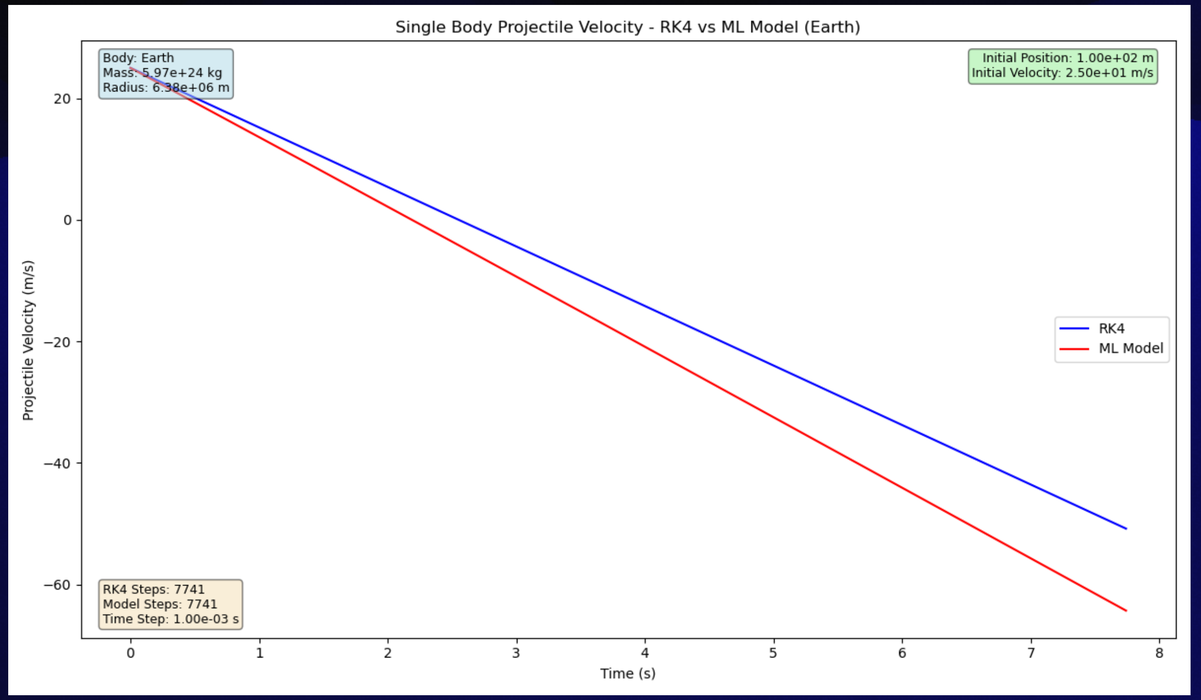


Prediction 1

Projectile launched from Earth, 2 training sessions

$$y_0 = 100\ (m), \qquad v_{y_0} = 25\ (m/s), \qquad t = 7.74\ (s), \qquad h = 0.001\ (s)$$

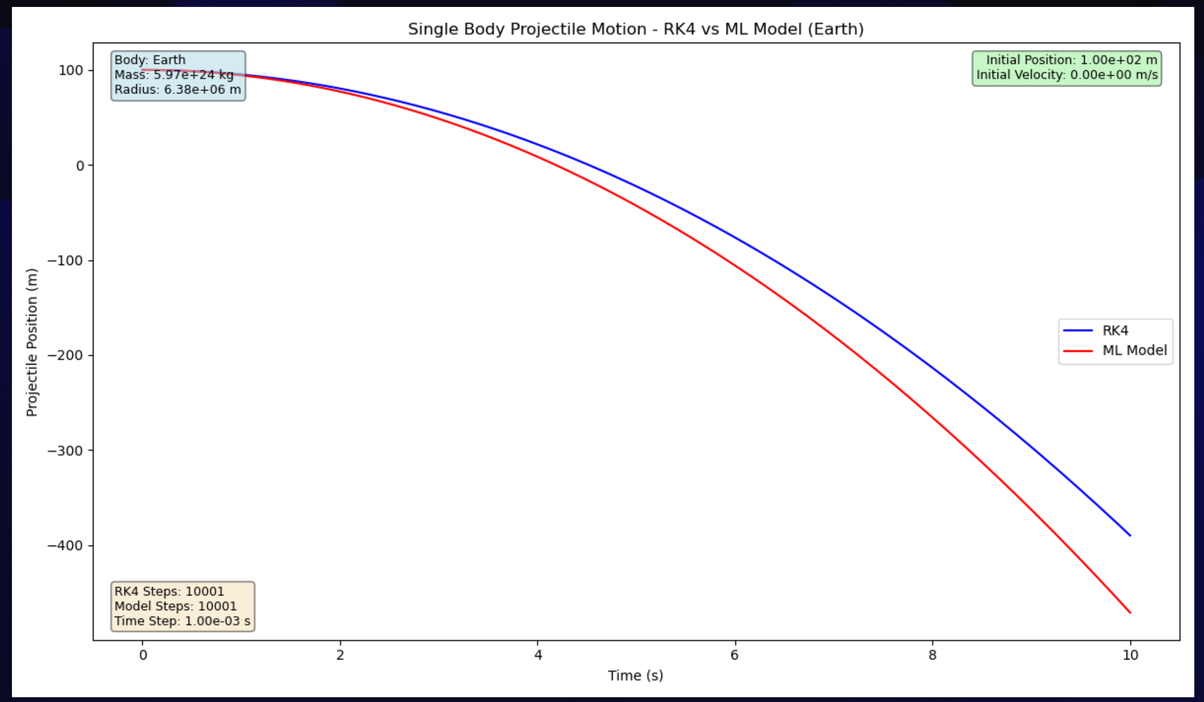


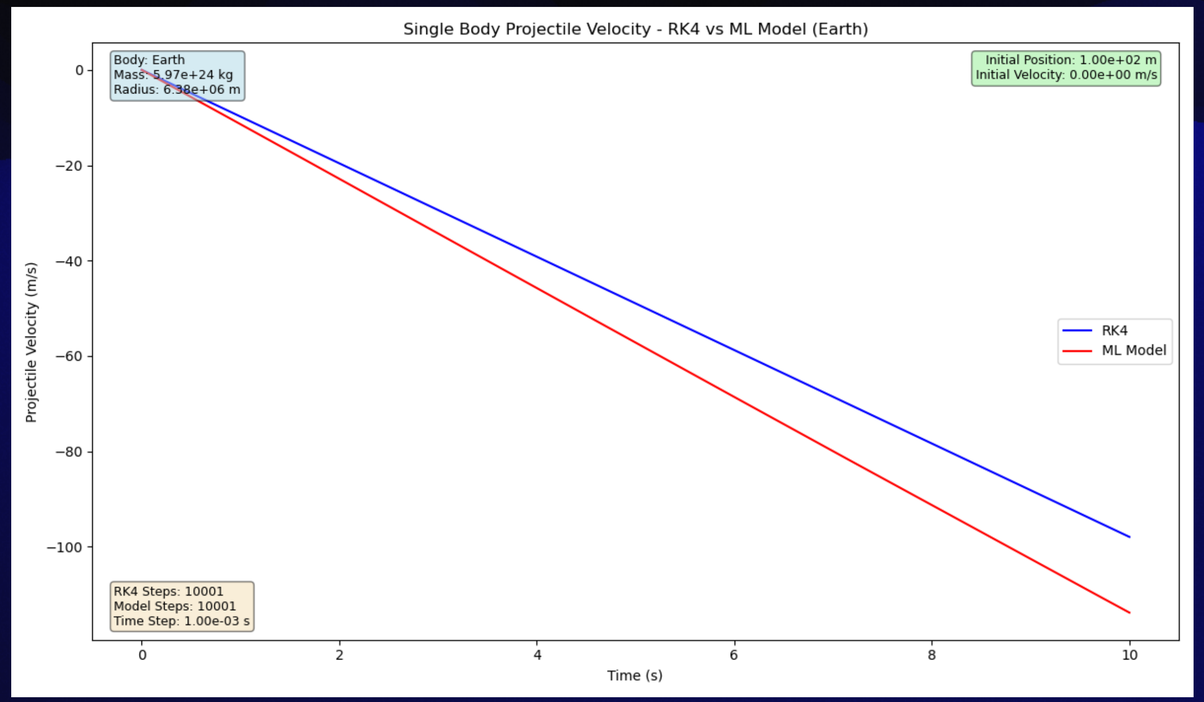


Prediction 2

Projectile launched from Earth, 1 training session

$$y_0 = 100\ (m), \qquad v_{y_0} = 0\ (m/s), \qquad t = 10.0\ (s), \qquad h = 0.001\ (s)$$

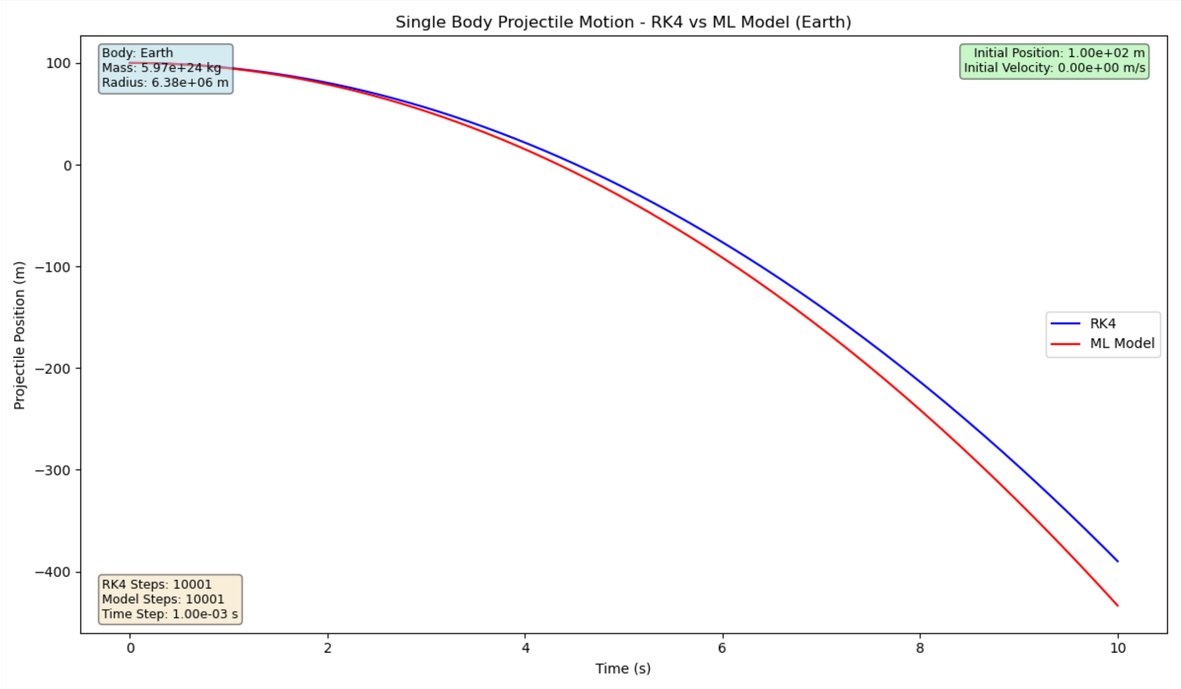


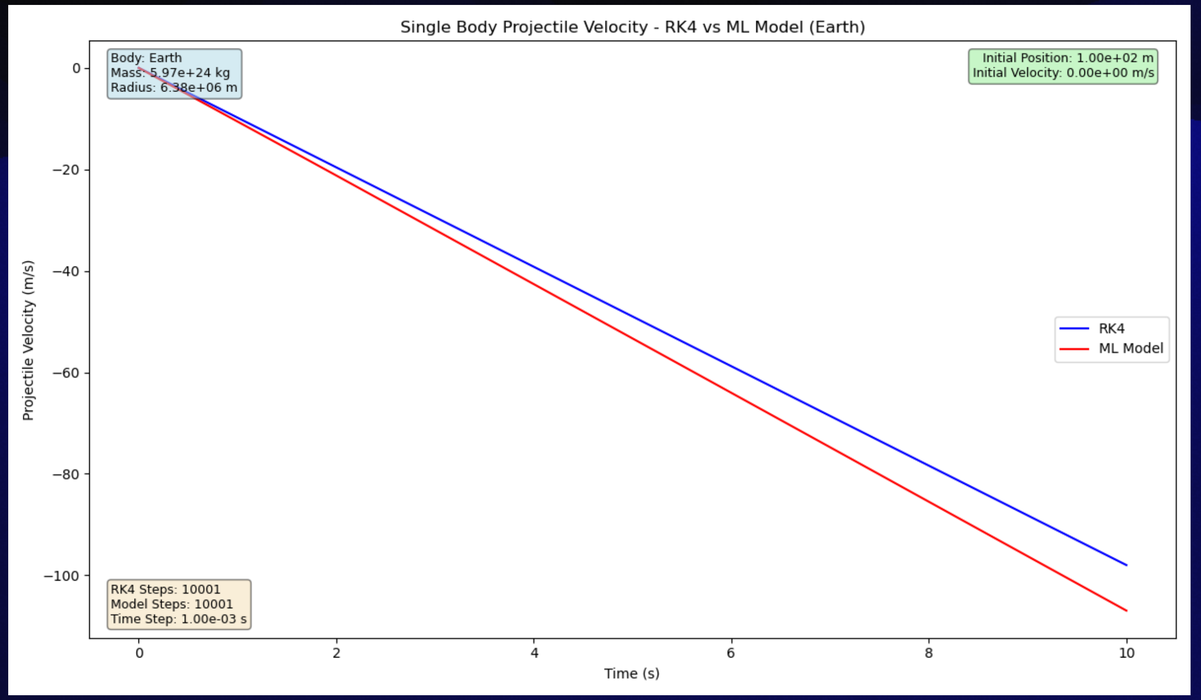


Prediction 2

Projectile launched from Earth, 2 training sessions

$$y_0 = 100\ (m), \qquad v_{y_0} = 0\ (m/s), \qquad t = 10.0\ (s), \qquad h = 0.001\ (s)$$

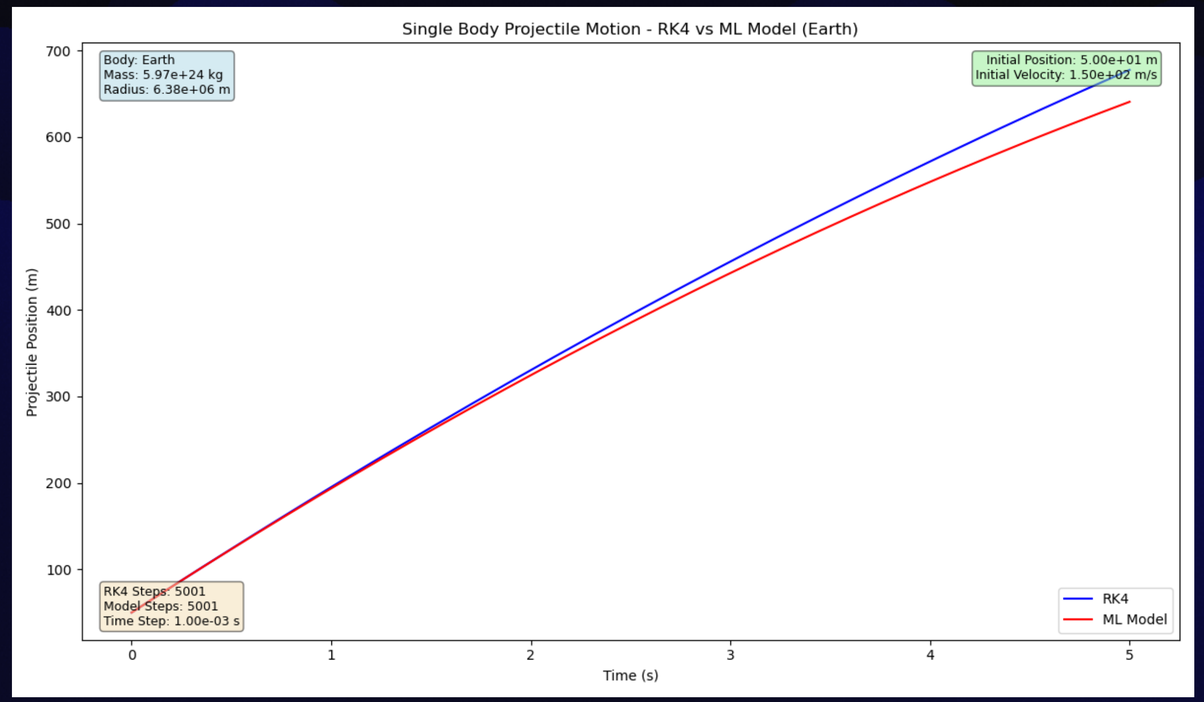


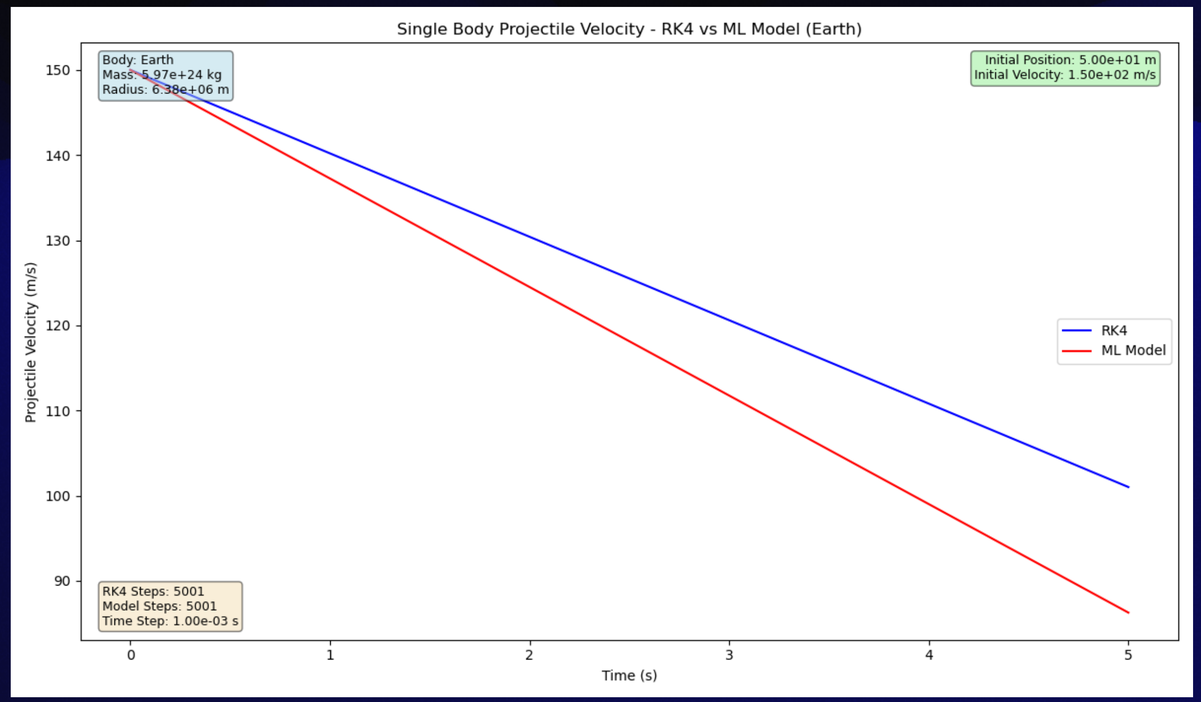


Prediction 3

Projectile launched from Earth, 2 training sessions

$$y_0 = 50\ (m), \qquad v_{y_0} = 150\ (m/s), \qquad t = 5.0\ (s), \qquad h = 0.001\ (s)$$

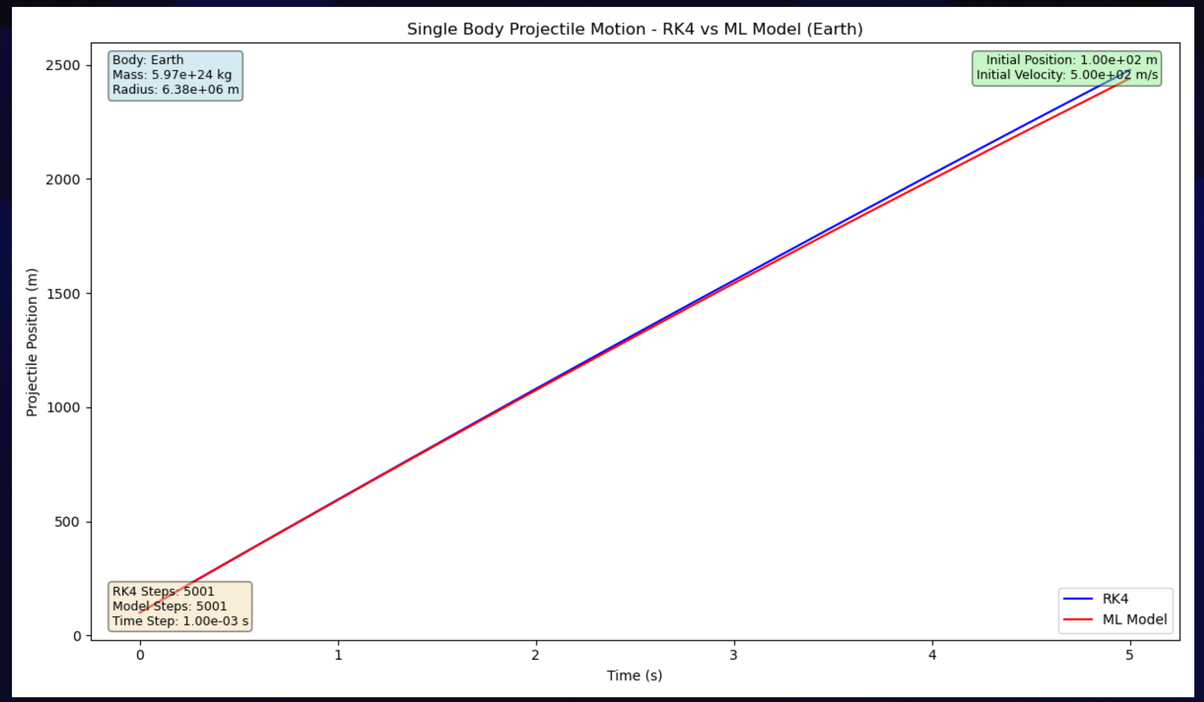


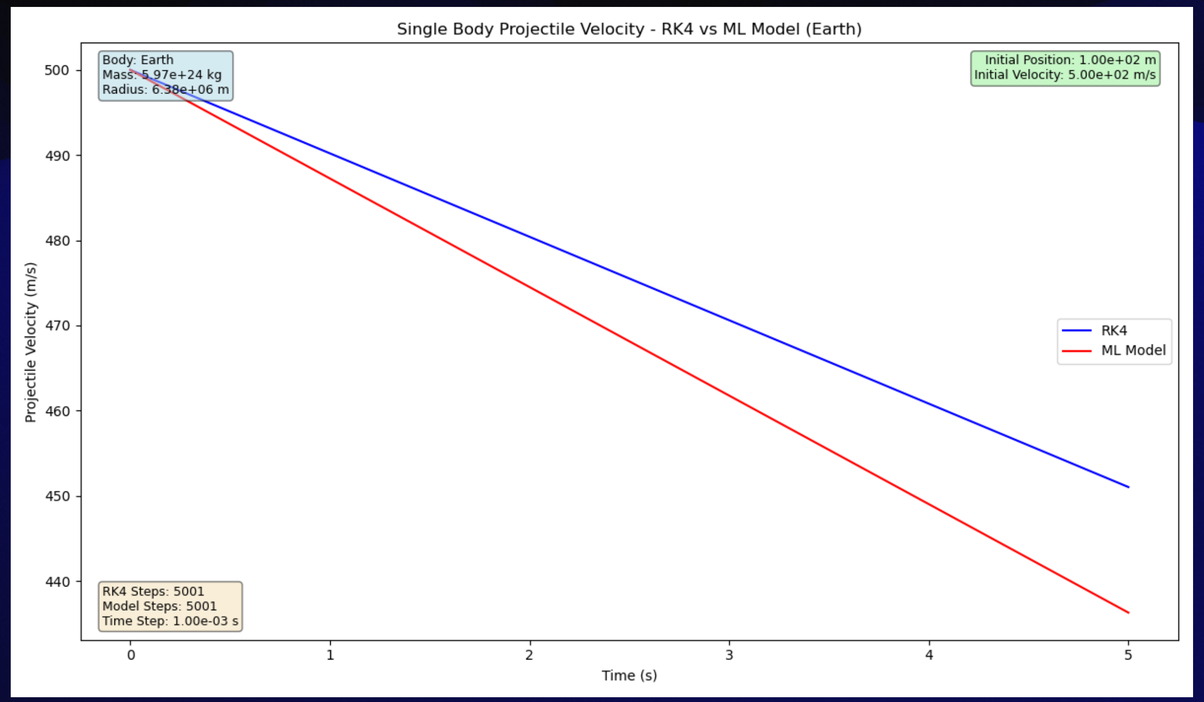


Prediction 4

Projectile launched from Earth, 2 training sessions

$$y_0 = 100\ (m), \qquad v_{y_0} = 500\ (m/s), \qquad t = 5.0\ (s), \qquad h = 0.001\ (s)$$

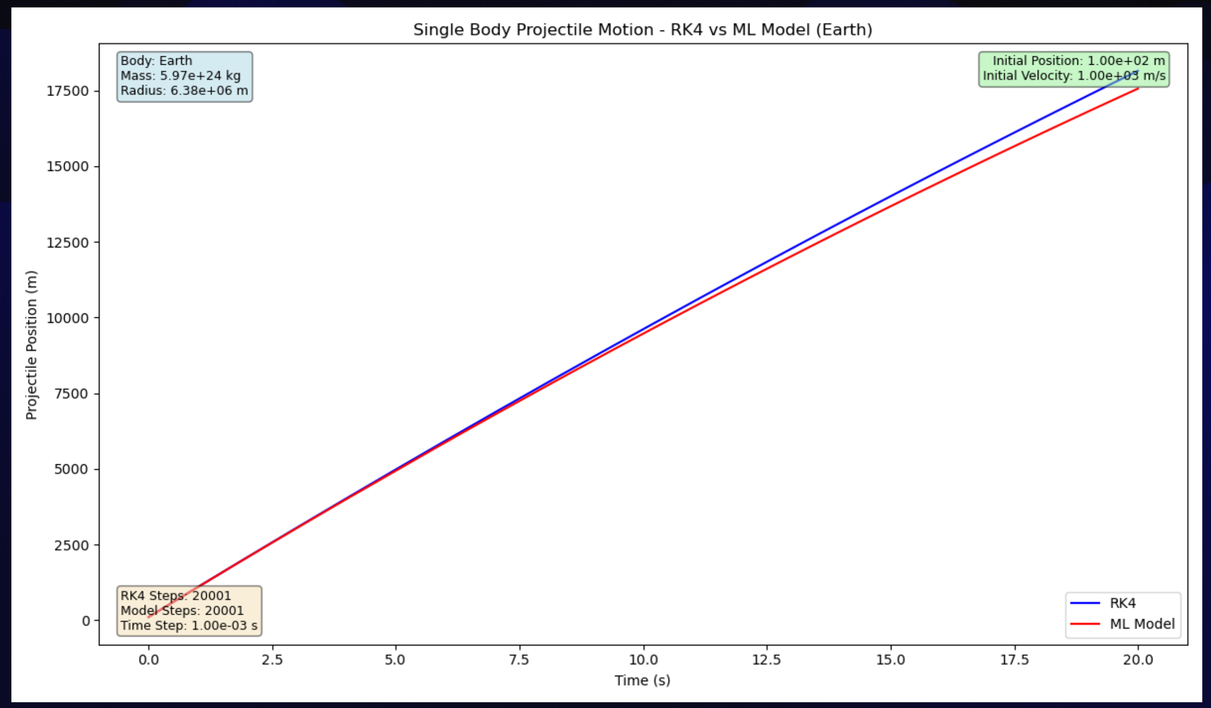


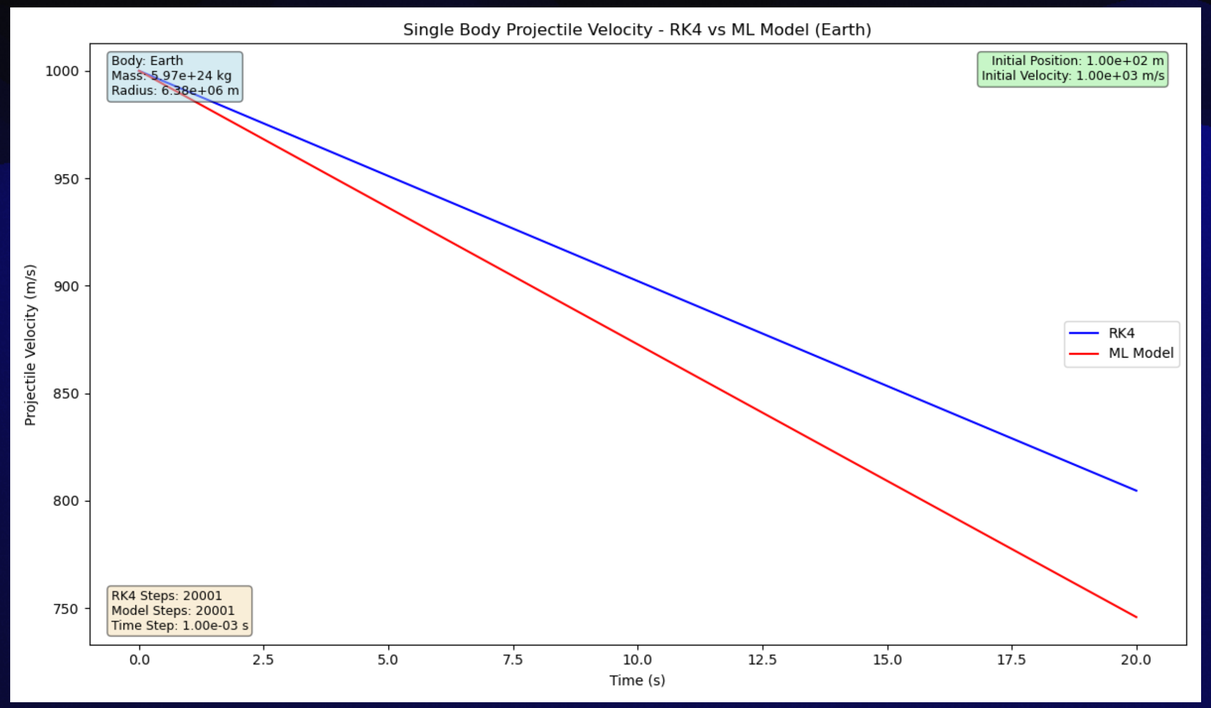


Prediction 5

Projectile launched from Earth, 2 training sessions

$$y_0 = 100\ (m), \qquad v_{y_0} = 1000\ (m/s), \qquad t = 20.0\ (s), \qquad h = 0.001\ (s)$$





Conclusions

Final Thoughts

Numerical Methods

- **RK4:** The Runge Kutta 4 method produces more stable trajectories / orbits compared to a more primitive method.
    - Method begins to struggle with higher number of bodies in N-Body System.

C++ vs. Python

- **C++:** Significantly faster, easier to bundle as a single executable, can do almost everything compared to Python.
    - C++ has a library called LibTorch for ML training and third party API called matplotlibcpp.
- **Python:** Very slow, has a harder time creating the same volume of data compared to C++.

Machine Learning

- **The Good:** Model training is in a good state, program can reliably train and retrain existing models with improvements over sessions.
- **The “Bad”**: There is currently only a model for the Single Body System, more code is needed for other systems.

Works Cited

- **Development:**
  - **GitHub Username:** QuantumCompiler
  - **GitHub Codebase:** https://github.com/QuantumCompiler/Dynamics-Of-Celestial-Bodies (Predicta-CLI)
  - **AI Companions:** ChatGPT and GitHub Copilot
  - **Projectile Motion PINN:** https://github.com/maciejczarnacki/PINNs-projectile-motion
- **Neat Stuff:**
  - **Kaggle:** https://www.kaggle.com/
  - **PINN Paper:** https://www.sciencedirect.com/science/article/abs/pii/S0021999118307125
- **Professional:**
  - **LinkedIn: Taylor Larrechea -** https://www.linkedin.com/in/tlarrechea/

Questions